Systems Programming Lecture Notes

시스템 프로그래밍 수업정리노트

CS:APP 기반 | SNU Byoungyoung Lee 교수 | 오픈북 시험 참고용

Ch1. Introduction — 시스템 지식이 왜 필요한가

과목의 관점: Programmer-Centric

이 과목은 컴퓨터 시스템을 **"사용하는 프로그래머"** 관점에서 바라본다. 대부분의 CS 과목은 추상화(Abstract Data Types, 점근 복잡도 등)를 강조하지만, 이 추상화 아래에 숨겨진 현실(Reality)을 모르면 심각한 버그와 성능 문제를 겪는다.

핵심 메시지: **"Abstraction is good, but don't forget reality"**

Five Great Realities

Reality #1: Int는 정수가 아니고, Float는 실수가 아니다

컴퓨터는 유한한 비트로 수를 표현하므로 수학적 성질이 깨진다.

```
// 오버플로우 예시
40000 * 40000 = 1,600,000,000  // OK (32-bit 범위 내)
50000 * 50000 = ???            // 오버플로우! 결과가 음수가 됨

// 결합법칙 위반 (Float)
(1e20 + -1e20) + 3.14 = 3.14      // 맞음
1e20 + (-1e20 + 3.14) = 0.0       // 틀림! 정밀도 손실
```

시험 포인트: signed integer overflow는 C에서 Undefined Behavior → 컴파일러가 임의로 코드 제거 가능

Reality #2: 어셈블리를 알아야 한다

직접 작성할 일은 없지만 이해는 필수다. - 버그가 왜 발생하는지 → 어셈블리 레벨 확인 필요 - 컴파일러가 어떤 최적화를 적용했는지 이해 - 보안 취약점(Buffer Overflow, ROP) 분석 - 역공학(proprietary software, malware 분석)

Reality #3: 메모리가 중요하다

- 메모리는 유한하고, C/C++은 메모리 접근을 **전혀 보호하지 않는다**

- 배열 경계 밖에 접근해도, 해제된 메모리에 접근해도 컴파일러가 막지 않음
- **공간적 버그**: 버그가 발생한 위치와 다른 위치의 데이터가 손상됨
- **시간적 버그**: 버그가 발생한 시점과 오류가 관측되는 시점이 다름

Reality #4: 성능 ≠ 점근적 복잡도

```c
// 동일한 O(n²) 알고리즘이지만 성능이 19배 차이!
void copyij(int src[2048][2048], int dst[2048][2048]) {
    for (i = 0; i < 2048; i++)
        for (j = 0; j < 2048; j++)
            dst[i][j] = src[i][j];  // 4.3ms ← 행 순서 접근
}

void copyji(int src[2048][2048], int dst[2048][2048]) {
    for (j = 0; j < 2048; j++)
        for (i = 0; i < 2048; i++)
            dst[i][j] = src[i][j];  // 81.8ms ← 열 순서 접근
}
```

이유: C 2D 배열은 Row-Major Order 저장 → 행 순서 접근이 캐시 지역성 ↑

Reality #5: I/O와 네트워크도 시스템의 일부다

파일 읽기, 네트워크 통신, 디스크 접근 모두 시스템 레벨에서 이루어지며, 이 I/O 성능이 프로그램 신뢰성과 성능에 결정적인 영향을 미친다.

Ch2-I. Machine-Level Programming: Basics

추상화 레벨

```
C 프로그래머:      소스 코드 (int x = a + b;)
                         ↓ gcc -S
어셈블리:          addq %rsi, %rdi
                         ↓ as
기계어:            0x48 0x01 0xFE
                         ↓ 하드웨어
컴퓨터 설계자:     게이트, 클록, 회로 레이아웃
```

ISA (Instruction Set Architecture): 프로그래머가 알아야 하는 부분 - 명령어 집합 명세, 레지스터

Microarchitecture: 프로그래머에게 보이지 않는 내부 구현 - 파이프라인, 캐시 크기, 분기 예측기, TLB

Programmer-Visible State (프로그래머가 볼 수 있는 상태)

```
PC (Program Counter) = %rip
  → 다음에 실행할 명령어 주소

레지스터 파일 (16개 64-bit 레지스터)
  → 자주 사용하는 데이터
```

```
Condition Codes (CF, ZF, SF, OF)
  → 최근 산술/논리 연산의 상태 정보
  → 조건 분기에 사용

Memory
  → 바이트 단위 주소 지정 가능한 배열
  → 코드, 데이터, 스택 포함
```

x86-64 레지스터 16개

```
%rax  ← 반환값 저장
%rbx  ← callee-saved
%rcx  ← 4번째 인자
%rdx  ← 3번째 인자
%rsi  ← 2번째 인자
%rdi  ← 1번째 인자
%rbp  ← callee-saved (frame pointer, 선택적)
%rsp  ← 스택 포인터 (항상 스택 top 가리킴)
%r8   ← 5번째 인자
%r9   ← 6번째 인자
%r10  ← caller-saved
%r11  ← caller-saved
%r12  ← callee-saved
%r13  ← callee-saved
%r14  ← callee-saved
%r15  ← callee-saved

각 레지스터의 하위 비트 접근:
%rax (64bit) / %eax (32bit) / %ax (16bit) / %al (8bit)
```

AT&T 어셈블리 문법

```
형식: [operation] [source], [destination]
     예: movq %rax, %rbx  → rbx = rax

접두사:
  $: 상수 (Immediate)    예: $0x400, $-533
  %: 레지스터           예: %rax, %r13
  없음: 메모리 참조      예: (%rax) = Mem[rax]

크기 접미사:
  b: 1 byte   w: 2 bytes   l: 4 bytes   q: 8 bytes
  예: movb, movw, movl, movq
```

Operand 3종류

```
1. Immediate (상수): $0x4, $-533
   → 값 자체

2. Register: %rax, %r13
   → 레지스터의 값

3. Memory: (%rax), 8(%rcx), (%rdx,%rcx,4)
   → 주소로 메모리에서 값을 읽음
```

```
★ 핵심: Memory → Memory 직접 이동 불가!
movq (%rax), (%rbx)  // 불법! 반드시 레지스터 경유해야 함
```

movq Operand 조합

Source	Dest	C 유사 표현
Imm	Reg	`temp = 0x4`
Imm	Mem	`*p = -147`
Reg	Reg	`temp2 = temp1`
Reg	Mem	`*p = temp`
Mem	Reg	`temp = *p`
Mem	Mem	**불가!**

메모리 주소 지정 일반형

```
D(Rb, Ri, S) = Mem[Reg[Rb] + S × Reg[Ri] + D]

  D:  변위(Displacement) — 1, 2, 4 bytes 상수
  Rb: 베이스 레지스터 — 16개 모두 가능
  Ri: 인덱스 레지스터 — %rsp 제외한 모든 레지스터
  S:  스케일 — 1, 2, 4, 8 중 하나

예시 계산 (%rdx=0xf000, %rcx=0x0100):
  0x8(%rdx)      → Mem[0xf000 + 8]          = Mem[0xf008]
  (%rdx,%rcx)    → Mem[0xf000 + 0x100]      = Mem[0xf100]
  (%rdx,%rcx,4)  → Mem[0xf000 + 4×0x100]   = Mem[0xf400]
  0x80(,%rdx,2)  → Mem[2×0xf000 + 0x80]    = Mem[0x1e080]
```

leaq vs movq — 핵심 차이

```
leaq 8(%rdi), %rax   // %rax = %rdi + 8  (주소값 저장, 메모리 접근 없음)
movq 8(%rdi), %rax   // %rax = Mem[%rdi+8]  (메모리에서 값 로드)
```

leaq의 실제 활용: 곱셈 최적화

```
long m3(long x) { return x * 3; }
// 컴파일 결과:
leaq (%rdi,%rdi,2), %rax   // %rax = %rdi + 2*%rdi = 3*%rdi
```

leaq는 메모리를 읽지 않고 주소 계산 자체를 목적으로 하는 명령어다. `x + k*y` 형태 (k=1,2,4,8)의 곱셈을 시프트+덧셈으로 최적화할 때 사용한다.

산술 연산 명령어

```
Two Operand:
  addq Src, Dest   → Dest = Dest + Src
  subq Src, Dest   → Dest = Dest - Src
  imulq Src, Dest  → Dest = Dest * Src
  salq Src, Dest   → Dest = Dest << Src  (좌시프트)
  sarq Src, Dest   → Dest = Dest >> Src  (산술 우시프트)
  shrq Src, Dest   → Dest = Dest >> Src  (논리 우시프트)
  xorq Src, Dest   → Dest = Dest ^ Src
  andq Src, Dest   → Dest = Dest & Src
```

```
  orq  Src, Dest   → Dest = Dest | Src

One Operand:
  incq Dest   → Dest = Dest + 1
  decq Dest   → Dest = Dest - 1
  negq Dest   → Dest = -Dest
  notq Dest   → Dest = ~Dest
```

Ch2-II. Machine-Level Programming: Control

Condition Codes (조건 코드)

CPU가 산술/논리 연산 시 자동으로 갱신하는 1-bit 플래그들:

```
CF (Carry Flag):    unsigned 오버플로우 발생 시 set
ZF (Zero Flag):     결과가 0일 때 set
SF (Sign Flag):     결과가 음수일 때 set (최상위 비트=1)
OF (Overflow Flag): signed 오버플로우 발생 시 set
                    (a>0 && b>0 && t<0) || (a<0 && b<0 && t>=0)
```

Condition Code 설정 방법

1. 암묵적 설정 (산술 연산의 부수 효과)

```
addq Src, Dest  →  t = a + b 계산 후 CF, ZF, SF, OF 자동 갱신
```

2. cmpq 명령어 (비교, 결과 저장 안함)

```
cmpq Src2, Src1
→ Src1 - Src2 계산 (저장하지 않음)
→ CF, ZF, SF, OF 갱신

★ 주의: AT&T 문법에서 순서는 cmpq b, a → a-b 계산
  cmpq %rsi, %rdi   →  %rdi - %rsi 계산
```

3. testq 명령어 (AND, 결과 저장 안함)

```
testq Src2, Src1
→ Src1 & Src2 계산 (저장하지 않음)
→ ZF, SF 갱신 (CF, OF는 항상 0으로 clear)

자주 쓰이는 패턴:
  testq %rax, %rax  →  ZF=1 if %rax==0, SF=1 if %rax<0
```

Conditional Jump 명령어

jX 명령어	조건	설명
jmp	1	무조건 점프
je	ZF	Equal / Zero
jne	~ZF	Not Equal / Not Zero
js	SF	Negative
jns	~SF	Nonnegative

```
jg            ~(SF^OF)&~ZF  Greater (signed)
jge           ~(SF^OF)      Greater or Equal (signed)
jl            SF^OF         Less (signed)
jle           (SF^OF)|ZF    Less or Equal (signed)
ja            ~CF&~ZF       Above (unsigned)
jb            CF            Below (unsigned)
```

조건 분기 예시

```c
long absdiff(long x, long y) {
    if (x > y) return x - y;
    else        return y - x;
}
```

```
absdiff:
    cmpq %rsi, %rdi      # x - y 비교
    jle  .L4             # if x <= y, goto L4
    movq %rdi, %rax
    subq %rsi, %rax      # return x - y
    ret
.L4:
    movq %rsi, %rax
    subq %rdi, %rax      # return y - x
    ret
```

루프 변환 패턴

컴파일러는 모든 루프를 **조건 점프 + 레이블**로 변환한다.

do-while → goto

```c
// 원본
do { body; } while (Test);

// 변환
loop:
    body
    if (Test) goto loop;
```

while → Jump-to-Middle (-Og 최적화)

```c
// 원본
while (Test) { body; }

// 변환 (Jump-to-Middle)
    goto test;
loop:
    body
test:
    if (Test) goto loop;
```

실제 예시: 비트 카운팅

```c
long pcount_while(unsigned long x) {
    long result = 0;
    while (x) {
        result += x & 0x1;
        x >>= 1;
    }
```

```
    return result;
}

// Jump-to-Middle 변환:
long pcount_goto_jtm(unsigned long x) {
    long result = 0;
    goto test;
loop:
    result += x & 0x1;
    x >>= 1;
test:
    if (x) goto loop;
    return result;
}

// 실제 어셈블리:
xorq %rax, %rax      # result = 0
.L2:  # loop:
    movq %rdi, %rdx
    andl $1, %edx    # t = x & 0x1
    addq %rdx, %rax  # result += t
    shrq %rdi        # x >>= 1
    jne  .L2         # if(x) goto loop
ret
```

Ch2-III. Machine-Level Programming: Procedures

함수 호출의 3가지 메커니즘

```
1. Passing Control (제어 전달)
   → 함수 시작점으로 이동
   → 반환 시 복귀 지점으로 이동

2. Passing Data (데이터 전달)
   → 인자 전달
   → 반환값 전달

3. Memory Management (메모리 관리)
   → 함수 실행 중 메모리 할당
   → 반환 시 해제
```

이 모든 것이 기계어 명령어로 구현된다. 구체적인 규칙은 **ABI (Application Binary Interface)**가 정의한다.

스택 구조와 push/pop

스택은 **높은 주소에서 낮은 주소 방향**으로 자란다.

```
높은 주소
┌──────────────┐
│              │  ← 스택 바닥 (높은 주소)
│              │
│    ...       │
```

```
│               │  ← %rsp 이전 (스택 내용)
├───────────────┤
│   현재 top    │  ← %rsp (항상 스택 top을 가리킴)
└───────────────┘
낮은 주소
```

pushq %rax (push 동작):

```
Step 1: %rsp -= 8
Step 2: Mem[%rsp] = %rax

동일한 효과:
    subq $8, %rsp
    movq %rax, (%rsp)
```

popq %rax (pop 동작):

```
Step 1: %rax = Mem[%rsp]
Step 2: %rsp += 8

동일한 효과:
    movq (%rsp), %rax
    addq $8, %rsp
```

call과 ret의 정확한 동작

call label:

```
Step 1: Push 복귀 주소 (call 다음 명령어의 주소)
Step 2: jmp label (목적 함수로 점프)

동일한 효과:
    pushq %rip_next   # 복귀 주소 스택에 저장
    jmp   label
```

ret:

```
Step 1: 스택 top의 주소를 %rip에 저장
Step 2: %rsp += 8

동일한 효과:
    popq  %tmp
    jmp   *%tmp
```

인자 전달 규칙 (Calling Convention)

```
인자 순서:  1번째  2번째  3번째  4번째  5번째  6번째  7번째~
레지스터:   %rdi   %rsi   %rdx   %rcx   %r8    %r9    스택(역순 push)
반환값:     %rax
```

7개 이상 인자 예시:

```
long f(long a1, long a2, long a3, long a4,
       long a5, long a6, long a7) {
    return a1 + a7;
}

// 어셈블리: a7은 스택으로 전달됨
// caller: pushq [a7], then call f
```

```
// callee: 8(%rsp)에서 a7 읽음 (ret addr 이후)
```

스택 프레임 레이아웃

```
호출 후 callee 함수 내 스택 상태:

높은 주소
┌──────────────────────────────┐
│   Caller의 7번째~ 인자        │  (있는 경우)
├──────────────────────────────┤
│   반환 주소 (Return Address) │  ← call 명령어가 push
├──────────────────────────────┤   ← (선택) 새 %rbp
│   Saved Callee Registers     │  ← callee-saved 레지스터 백업
│   (%rbx, %r12~%r14, %rbp)   │
├──────────────────────────────┤
│   지역 변수 (Local Vars)     │
├──────────────────────────────┤
│   인자 빌드 공간              │  (다음 호출용 7번째~ 인자)
└──────────────────────────────┘   ← %rsp
낮은 주소
```

레지스터 저장 규칙

왜 필요한가? 함수 A가 레지스터를 사용하다가 함수 B를 호출하면, B도 같은 레지스터를 사용할 수 있다. 서로 약속이 필요하다.

Caller-saved (호출자 저장):

```
레지스터: %rax, %rcx, %rdx, %rsi, %rdi, %r8~%r11
규칙: Callee가 자유롭게 덮어쓸 수 있음
     필요하다면 Caller가 call 전에 직접 저장해야 함

왜 이 이름?: "Caller가 책임지고 저장해야 한다"
```

Callee-saved (피호출자 저장):

```
레지스터: %rbx, %r12~%r14, %rbp
규칙: Callee는 이 레지스터를 사용하기 전에 push로 저장,
     반환하기 전에 pop으로 복원해야 함

왜 이 이름?: "Callee가 책임지고 원래 값을 보존해야 한다"
```

실제 예시 — 재귀 함수:

```
long pcount(unsigned long x) {
    if (x == 0) return 0;
    return (x & 1) + pcount(x >> 1);
}

pcount:
    pushq %rbx          # %rbx는 callee-saved → 저장
    movq  %rdi, %rbx    # x를 %rbx에 보관 (재귀 호출 후에도 필요)
    movl  $0, %eax
    testq %rdi, %rdi
    je    .L6
    shrq  %rdi          # x >> 1
    call  pcount        # 재귀 호출 → %rbx는 보존됨
    andl  $1, %ebx      # x & 1 (재귀 후에도 %rbx에 있음)
    addq  %rbx, %rax    # (x&1) + pcount(x>>1)
```

```
.L6:
    popq  %rbx          # %rbx 복원
    ret
```

핵심: `x & 1` 값을 재귀 호출 이후에도 사용해야 하므로 caller-saved 레지스터(%rdi 등)가 아닌 callee-saved 레지스터(%rbx)에 저장한다.

Ch3. Stack Buffer Overflow

x86-64 Linux 메모리 레이아웃

```
0xFFFFFFFFFFFFFFFF (높은 주소)
┌──────────────────────┐
│       커널 영역       │  (사용자 접근 불가)
├──────────────────────┤ 0x7FFFFFFFFFFFFFFF
│                      │
│   Stack (↓ 성장)     │ ← 지역 변수, 반환 주소
│                      │
├──────────────────────┤
│                      │
│   Shared Libraries   │ ← libc.so 등 (ASLR로 랜덤화)
│                      │
├──────────────────────┤
│                      │
│   Heap (↑ 성장)      │ ← malloc, calloc, new
│                      │
├──────────────────────┤ brk 포인터
│   Data (.data/.bss)  │  ← 전역변수, static 변수
├──────────────────────┤
│   Text (.text)       │  ← 실행 코드 (read-only)
└──────────────────────┘ 0x0000000000400000
```

Buffer Overflow 취약점 원리

```
void copy_name(char *src) {
    char name[32];      // ← rbp - 0x20 위치에 배치
    strcpy(name, src);  // 길이 검사 없음!
}
```

스택 레이아웃:

```
copy_name() 호출 시:

높은 주소
┌──────────────────────┐
│   반환 주소 (8 bytes) │  ← name + 40
├──────────────────────┤
│   saved %rbp (8 bytes)│  ← name + 32
├──────────────────────┤ ← %rbp
│   name[0..31]        │  ← rbp - 0x20 = name 시작점
└──────────────────────┘ ← %rsp
낮은 주소

name에 33바이트 이상 쓰면 → saved %rbp 오염
name에 41바이트 이상 쓰면 → 반환 주소 오염
```

오염 후 동작:

```
ret 명령어 실행:
  popq %rip  ← 오염된 반환 주소를 %rip에 로드
  → 공격자가 원하는 주소로 점프!
```

Buffer Overflow 공격 3가지

1. Return Address Overwrite (반환 주소 덮어쓰기)

```
name[] = [padding 40 bytes] + [원하는 함수 주소]
→ 함수 반환 시 다른 함수(예: print_passwd)로 점프
```

2. Code Injection (코드 삽입)

```
name[] = [악성 기계어 코드(shellcode)] + [name 배열 시작 주소]
→ 반환 주소가 name[] 시작을 가리킴
→ ret 실행 시 shellcode 실행

shellcode 예시 (bash 실행):
\x48\xbb\x2f\x62\x69\x6e\x2f\x73\x68\x00  # mov /bin/sh
\x53\x48\x89\xe7\x31\xf6\x31\xd2\xb8\x3b  # syscall execve
\x00\x00\x00\x0f\x05
```

3. Return-Oriented Programming (ROP)

```
NX Bit가 스택 실행을 막으면 → 기존 코드를 활용!

Gadget = ret으로 끝나는 기존 코드 시퀀스
예:
  0x4005a1: pop %rdi ; ret    ← Gadget 1
  0x4005b3: pop %rsi ; ret    ← Gadget 2
  0x4005cd: syscall           ← Gadget 3

스택에 Gadget 주소들을 연결:
  [Gadget1 주소] [인자1] [Gadget2 주소] [인자2] [Gadget3 주소]

ret 체인:
  ret → Gadget1 실행 → ret → Gadget2 실행 → ret → ...
  → NX 우회! (실행되는 코드는 이미 .text에 있는 코드)
```

Buffer Overflow 방어 3+1가지

1. 안전한 코딩

```
// 위험한 함수들
gets(buf)           // 길이 검사 없음
strcpy(dst, src)    // 길이 검사 없음
scanf("%s", buf)    // 길이 검사 없음

// 안전한 대안
fgets(buf, sizeof(buf), stdin)  // 최대 길이 지정
strncpy(dst, src, n)           // 최대 길이 지정
```

2. Stack Canary (스택 카나리)

```
함수 진입 시:
  %fs:0x28 → 전역 랜덤값을 스택에 복사
  (버퍼와 반환 주소 사이에 배치)
```

```
함수 반환 전:
  저장된 canary 값 == %fs:0x28 확인
  불일치 시 → 즉시 프로그램 종료

gcc -fstack-protector (현재 기본 옵션)

어셈블리에서의 모습:
  mov %fs:0x28, %rax
  mov %rax, -0x8(%rbp)   # 스택에 canary 저장
  ...
  mov -0x8(%rbp), %rax
  xor %fs:0x28, %rax     # canary 비교
  jne .L_stack_fail      # 불일치 시 fail
```

3. ASLR (Address Space Layout Randomization)

```
매 실행마다 스택, 힙, 공유 라이브러리의 시작 주소를 랜덤화

같은 프로그램 5번 실행:
  실행1: 스택 시작 = 0x7ffe4d3be000
  실행2: 스택 시작 = 0x7fff75a4f000
  실행3: 스택 시작 = 0x7ffeadb7c000
  ...

→ 공격자가 shellcode 주소, Gadget 주소를 예측 불가
```

4. NX Bit (Non-Execute Bit)

```
메모리 영역별 권한 설정:
  .text 영역:  읽기(R) + 실행(X) 가능,  쓰기(W) 불가
  스택 영역:   읽기(R) + 쓰기(W) 가능, 실행(X) 불가  ← NX Bit
  힙 영역:     읽기(R) + 쓰기(W) 가능, 실행(X) 불가

PTE의 XD (eXecute Disable) 비트로 구현
→ Code Injection 방어 (스택의 shellcode 실행 불가)
→ ROP로 우회 가능 (기존 코드 활용이므로 NX 관계 없음)
```

공격-방어 관계:

```
Code Injection → NX Bit로 방어
NX Bit 우회   → ROP 사용
ROP           → ASLR로 Gadget 주소 예측 어렵게
Stack 오버플로우 → Stack Canary로 탐지
```

Ch4. Memory Hierarchy

CPU-Memory Gap (메모리 격차 문제)

```
CPU 속도:  GHz 단위 (클록 당 수 nanoseconds)
DRAM 접근: ~100 nanoseconds → CPU보다 100배 이상 느림
HDD 접근:  ~10,000,000 nanoseconds → CPU보다 10,000,000배 느림

이 격차가 매년 벌어지고 있다 → 해결책: 메모리 계층 구조
```

지역성 (Locality) — 캐시가 동작하는 이유

프로그램은 메모리에 무작위로 접근하지 않는다. 두 가지 패턴이 있다.

시간적 지역성 (Temporal Locality):

```
최근에 접근한 데이터는 곧 다시 접근할 가능성이 높다.

예: for 루프의 인덱스 변수 i, 누적 합계 sum
  for (int i = 0; i < n; i++)
      sum += a[i];   // sum은 매 반복 접근 → 시간적 지역성 ↑
```

공간적 지역성 (Spatial Locality):

```
최근에 접근한 주소 근처를 곧 접근할 가능성이 높다.

예: 배열 순차 접근
  for (int i = 0; i < n; i++)
      sum += a[i];   // a[0], a[1], a[2], ... → 공간적 지역성 ↑
```

Row-Major Order와 성능 영향

C의 2D 배열은 **행 우선(Row-Major)** 방식으로 메모리에 저장된다:

```
int a[M][N]:
메모리: a[0][0], a[0][1], ..., a[0][N-1], a[1][0], a[1][1], ...
        ←──────── Row 0 ────────→ ←──────── Row 1 ────────→
```

좋은 패턴 (행 순서 접근):

```
int sum_array_rows(int a[M][N]) {
    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            sum += a[i][j];   // a[0][0], a[0][1], ... 순차 접근
}
// → 공간적 지역성 ↑ → Cache Hit ↑ → 빠름 (4.3ms)
```

나쁜 패턴 (열 순서 접근):

```
int sum_array_cols(int a[M][N]) {
    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            sum += a[i][j];   // a[0][0], a[1][0], a[2][0], ... N칸씩
건너뜀
}
// → 공간적 지역성 ↓ → Cache Miss ↑ → 느림 (81.8ms, 19배 차이)
```

왜 19배 차이?

```
Cache Hit:  ~4 cycles
Cache Miss: ~100 cycles (DRAM 접근 필요)
Cache Line: 64 bytes

행 접근: 64/4 = 16개 int마다 1번 miss → Hit 비율 매우 높음
열 접근: 2048 * 4 = 8192 bytes씩 건너뜀 → 매번 miss
```

메모리 계층 구조

접근 시간	크기	레벨	위치
0 cycles	수십 바이트	L0	CPU 레지스터

```
4 cycles    32 KB         L1       CPU 내 SRAM
10 cycles   256 KB        L2       CPU 내 SRAM
수십 cycles  8 MB          L3       CPU 내 SRAM (코어 공유)
100 cycles  수십 GB       L4       DRAM (Main Memory)
10^7 cycles 수 TB         L5       SSD / HDD (Local Disk)
10^9 cycles 수 PB         L6       원격 서버 (Network)
```

계층의 원칙: 각 레벨 k는 레벨 k+1의 캐시 역할을 한다. - 지역성 덕분에 프로그램은 레벨 k+1보다 레벨 k의 데이터를 더 자주 접근한다.

Cache 동작 원리

```
데이터 단위: Cache Line = 64 bytes (x86-64)
메모리는 Cache Line 단위로 블록화되어 있다.

Cache Hit:
  CPU가 주소 A에 접근 → 해당 블록이 캐시에 있음
  → 캐시에서 직접 반환 (빠름)

Cache Miss:
  CPU가 주소 A에 접근 → 해당 블록이 캐시에 없음
  → 다음 레벨 메모리에서 블록 전체(64 bytes) 로드
  → Placement Policy: 어느 캐시 라인에 넣을지 결정
  → Replacement Policy: 기존 캐시 내용을 어떤 것을 교체할지
     (LRU, FIFO, Random 등)
  → 요청한 데이터 반환
```

Ch5. Code Optimization & Linking

컴파일러 최적화의 목표와 원칙

목표: 1. 명령어 수 최소화 (불필요한 계산 제거) 2. 메모리 접근 최소화 (레지스터 최대 활용) 3. 분기 최소화 (CPU 예측 실패 비용 제거)

핵심 제약: 프로그램 동작이 바뀌어서는 안 된다 (관측 가능한 부수 효과 보존)

주요 최적화 기법

1. Constant Folding (상수 접기)

```
long mask = 0xFF << 8;  →  long mask = 0xFF00;  // 컴파일 시간에 계산
size_t n = strlen("Harry Bovik");  →  size_t n = 11;
```

2. Strength Reduction (연산 강도 감소)

```
long a = b * 5;  →  long a = (b << 2) + b;  // 곱셈 → 시프트+덧셈
// 곱셈은 수십 cycles, 시프트+덧셈은 1-2 cycles
```

3. Dead Code Elimination (죽은 코드 제거)

```
if (0) { puts("Kilroy was here"); }   // 절대 실행 안됨 → 제거
x = 0;
x = 23;  // 이전 x=0 대입은 무의미 → 제거
```

4. Common Subexpression Elimination (공통 부분식 제거)

```
// Before:
norm[i] = v[i].x * v[i].x + v[i].y * v[i].y;

// After (CSE 적용):
elem = &v[i];
x = elem->x; y = elem->y;
norm[i] = x*x + y*y;
// → v[i] 접근 6번 → 2번으로 감소
```

5. Code Motion (코드 이동)

```
// Before:
for (j = 0; j < n; j++)
    a[n*i+j] = b[j];   // n*i가 매 반복 계산됨

// After (루프 불변 계산을 루프 밖으로):
int ni = n * i;          // 한 번만 계산
for (j = 0; j < n; j++)
    a[ni+j] = b[j];
```

6. Inlining (인라인화)

```
// Before:
int func(int y) {
    return pred(y) + pred(0) + pred(y+1);
}

// After (인라인 + 추가 최적화):
int func(int y) {
    int tmp = 0;
    if (y != 0) tmp = y - 1;     // pred(y) 인라인
    // pred(0) → 상수 0 (constant folding)
    if (y != -1) tmp += y;        // pred(y+1) 인라인 + 단순화
    return tmp;
}
```

7. Loop Unrolling (루프 전개)

```
// Before:
for (i = 0; i < n; i++)
    A[i] = B[i]*k + C[i];

// After (4x unrolling):
for (i = 0; i < n - 4; i += 4) {
    A[i]   = B[i]*k   + C[i];
    A[i+1] = B[i+1]*k + C[i+1];
    A[i+2] = B[i+2]*k + C[i+2];
    A[i+3] = B[i+3]*k + C[i+3];
}
// → 루프 조건 검사 횟수 4배 감소
```

컴파일러 최적화의 한계

왜 `lower1`을 `lower2`로 최적화할 수 없는가?

```
void lower1(char *s) {
    for (size_t i = 0; i < strlen(s); i++)  // O(n²)!
        if (s[i] >= 'A' && s[i] <= 'Z')
```

```
            s[i] -= 32;
}

void lower2(char *s) {
    size_t n = strlen(s);  // 한 번만 호출 → O(n)
    for (size_t i = 0; i < n; i++)
        if (s[i] >= 'A' && s[i] <= 'Z')
            s[i] -= 32;
}
```

컴파일러가 `strlen(s)`를 루프 밖으로 이동시키지 못하는 이유: - `s[i] -= 32` 동작이 `s`를 수정함 - `strlen` 함수는 `s`를 읽으므로, 내부 루프에서 `s`가 변경되면 `strlen` 결과도 달라질 수 있음 - 컴파일러는 안전을 위해 이를 루프 밖으로 이동하지 않음

Undefined Behavior 위험:

```
long long_cmp_opt(const int a, const int b) {
    if (a > 0) {
        do_something();
        if (b < 0) {
            do_something_else();
            if ((a - b) > 0)    // Q: 이 조건이 항상 true인가?
                do_another_thing();
        }
    }
}
```

수학적으로는: a > 0, b < 0이면 a-b > 0이 항상 성립 하지만 C에서 signed integer overflow는 **Undefined Behavior** → 컴파일러는 "이 코드는 UB가 없다고 가정" → 조건 검사 자체를 제거 가능! → 실제로 Linux 커널에서 이 패턴으로 심각한 버그 발생 (lwn.net/Articles/511259)

링킹 (Linking)

링킹은 여러 `.o` 파일을 하나의 실행 파일로 합치는 과정이다.

링킹의 3단계:

```
Step 1: 심볼 테이블 구성 (어셈블러가 수행)
──────────────────────────────────────────

각 .o 파일은 심볼 테이블을 포함:
  Global 심볼: 다른 파일에서 참조 가능 (non-static 함수/전역변수)
  Local 심볼:  같은 파일 내에서만 (static 함수/변수)
  External Reference: 다른 파일에서 정의된 심볼 참조

예: main.c가 sum() 함수를 호출 → sum은 external reference


Step 2: 심볼 해석 (링커가 수행)
──────────────────────────────────────────

각 External Reference를 정의(Definition)에 연결
  main.o의 sum 참조 → sum.o의 sum 정의에 연결


Step 3: 재배치 (링커가 수행)
──────────────────────────────────────────

.o 파일의 임시 주소(0x0) → 최종 실행 파일의 절대 주소로 교체

재배치 전 (main.o):
```

```
  0: sub $0x8, %rsp
  4: mov $0x2, %esi
  9: mov $0x0, %edi    # &array = 0x0 (임시 플레이스홀더)
  a: R_X86_64_32 array  ← 재배치 엔트리
  e: call 13            # sum() = 임시 오프셋
  f: R_X86_64_PC32 sum  ← 재배치 엔트리

재배치 후 (prog):
  4004d9: mov $0x601018, %edi  # &array = 0x601018 (절대 주소)
  4004de: call 4004e8 <sum>    # sum()의 실제 주소
```

Static Library vs Dynamic Library

Static Library (.a):

```
ar rs libc.a atoi.o printf.o random.o ...

특징:
+ 링크 시 필요한 부분만 실행 파일에 포함
+ 실행 파일이 독립적 (라이브러리 불필요)
- 모든 실행 파일이 같은 코드를 중복 포함
- 라이브러리 버그 수정 시 모든 실행 파일 재링크 필요
```

Dynamic/Shared Library (.so):

```
gcc -shared -o libm.so ...

특징:
+ 여러 프로세스가 같은 물리 메모리를 공유 (메모리 절약)
+ 라이브러리 업데이트 시 실행 파일 재링크 불필요
- 실행 시 라이브러리 파일 필요
- Load-time linking: 프로그램 시작 시 자동 링크 (ld-linux.so)
- Run-time linking: 실행 중 dlopen()으로 명시적 링크

ldd로 의존 라이브러리 확인:
$ ldd prog
  linux-vdso.so.1 => (0x00007ffcf2998000)
  libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
```

시스템 프로그래밍 수업정리노트 (Part 2: Ch6~Ch10)

Ch6. Virtual Memory (가상 메모리)

왜 가상 메모리가 필요한가?

```
문제 상황: 여러 프로세스가 동시에 실행 중
  Process 1: 0x400000 ~ 0x600000 사용
  Process 2: 0x400000 ~ 0x600000 사용  ← 주소 충돌!

해결책: 각 프로세스에게 가상 주소 공간 제공
  → 모두 0x400000부터 시작하는 것처럼 보이지만
```

```
  → 실제 물리 주소는 다름 (MMU가 변환)
```

Physical Addressing (물리 주소 방식): - 임베디드 시스템, 간단한 마이크로컨트롤러에서 사용 - CPU가 생성한 주소가 바로 물리 메모리 주소

Virtual Addressing (가상 주소 방식): - 모든 현대 OS(Linux, Windows, macOS)에서 사용 - CPU가 생성한 가상 주소 → MMU가 물리 주소로 변환

VM의 3가지 역할

역할 1: Caching (캐싱)

```
모든 데이터는 원래 디스크(또는 파일 시스템)에 있다.
자주 접근하는 데이터(페이지)를 DRAM에 캐시한다.

               디스크 (원본 데이터)
                       ↕ (Page Fault 발생 시 로드)
  가상 메모리 → 페이지 테이블 → DRAM (물리 메모리)
  (프로세스 뷰)              (실제 저장 위치)

페이지(Page): DRAM 캐시의 기본 단위 = 4KB (2¹² bytes)
```

역할 2: Memory Management (메모리 관리)

```
각 프로세스에게 동일한 형태의 선형 가상 주소 공간 제공

Process 1:  0 ~ 2^64-1 (가상 주소 공간)
Process 2:  0 ~ 2^64-1 (가상 주소 공간)
                ↓ 페이지 테이블을 통해
물리 메모리: 실제로는 서로 다른 물리 페이지에 매핑

혜택:
- 연속된 가상 주소 ≠ 연속된 물리 주소 (단편화 방지)
- 코드/라이브러리 공유 가능 (여러 가상 → 동일 물리)
- fork() 시 COW로 효율적 복제
```

역할 3: Memory Protection (메모리 보호)

```
PTE (Page Table Entry)의 권한 비트:
  R/W: 읽기 전용 or 읽기/쓰기
  U/S: 유저 모드 접근 가능 or 슈퍼바이저(커널) 전용
  XD:  실행 불가 (NX Bit)

MMU는 매 메모리 접근 시 이 비트를 검사
→ 커널 메모리: U/S=0 → 유저 프로세스 접근 시 예외 발생
→ 읽기 전용 페이지: R/W=0 → 쓰기 시 Protection Fault
```

페이지 테이블 (Page Table)

```
페이지 테이블: VPN → PPN 매핑 테이블
  - 프로세스마다 독립적으로 존재
  - 커널이 DRAM에 유지
  - CR3 레지스터: 현재 프로세스의 L1 페이지 테이블 물리 주소

PTE (Page Table Entry) 구조:
  비트 63:   XD (eXecute Disable, NX Bit)
```

```
  비트 51-12: PPN (Physical Page Number, 40 bits)
  비트 8:    G (Global)
  비트 7:    PS (Page Size)
  비트 6:    D (Dirty) - 해당 페이지에 쓰기 발생 여부
  비트 5:    A (Accessed) - 읽기/쓰기 발생 여부
  비트 4:    CD (Cache Disable)
  비트 3:    WT (Write Through)
  비트 2:    U/S (User/Supervisor)
  비트 1:    R/W (Read/Write)
  비트 0:    P (Present) - DRAM에 현재 존재 여부
```

가상 주소 → 물리 주소 변환

```
가상 주소 (48-bit):
  ┌──────────────────────────────────────┬──────────┐
  │          VPN (36 bits)               │ VPO(12 bits)│
  └──────────────────────────────────────┴──────────┘


페이지 테이블 조회:
  PTEAddress = PageTableBase + VPN * sizeof(PTE)
  PTE = Mem[PTEAddress]
  if PTE.P == 0: Page Fault!
  PPN = PTE[51:12]

물리 주소 (52-bit):
  ┌────────────────────┬──────────┐
  │    PPN (40 bits)   │ PPO(12 bits)│
  └────────────────────┴──────────┘
  여기서 PPO = VPO (동일)
```

Page Fault 처리 7단계 (★ 시험 필수)

```
① CPU가 VA에 메모리 접근 → MMU가 PTE 조회
② PTE의 P(Present) bit = 0 → Page Fault 예외 발생
   (또는 권한 위반이면 Protection Fault)
③ 커널의 Page Fault Handler로 제어 전달
④ Handler가 교체할 Victim 페이지 선택 (LRU 등)
⑤ Victim 페이지의 D(Dirty) bit = 1이면 → 디스크에 먼저 기록 (Write-back)
⑥ 필요한 페이지를 디스크에서 물리 메모리로 로드
⑦ PTE 갱신 (P=1, PPN 설정) → `iret` 실행 → 원래 명령어 재실행 (이번엔
Page Hit)

iret: 인터럽트/폴트 반환 명령어
  = ret + 권한 레벨(Ring Level) 복원
```

Thrashing (스래싱)

```
Working Set: 프로그램이 특정 시점에 활발히 접근하는 페이지 집합

if (Working Set 크기 < DRAM 크기):
    → 좋은 성능 (대부분 Page Hit)

if (Working Set 크기 > DRAM 크기):
    → Thrashing 발생!
    → 페이지를 로드했다 내쫓았다를 반복
    → 프로그램이 거의 멈춘 것처럼 보임
```

```
여러 프로세스 동시 실행 시:
    sum(Working Sets) > DRAM → Thrashing
```

TLB (Translation Lookaside Buffer)

문제: 페이지 테이블이 DRAM에 있으므로, 매 메모리 접근마다: 1. PTE를 읽기 위해 DRAM 접근 (1회) 2. 실제 데이터를 읽기 위해 DRAM 접근 (1회) → 총 2회의 DRAM 접근 필요 → 느림!

해결: TLB = MMU 내부의 소형 PTE 캐시

```
TLB Hit (대부분의 경우):
  VA → TLB에서 VPN → PPN 변환 → PA → 캐시/메모리
  DRAM 접근: 1회 (PTE 조회 불필요)

TLB Miss (드문 경우):
  VA → TLB Miss → DRAM에서 PTE 조회 → TLB 갱신 → PA
  DRAM 접근: 2회

TLB Miss가 드문 이유: 시간적 지역성
  → 최근에 변환한 페이지는 곧 다시 접근할 가능성이 높음
```

Multi-Level Page Table

문제: 단일 페이지 테이블의 크기

```
48-bit VA, 4KB 페이지, 8 bytes PTE:
  페이지 수 = 2^48 / 2^12 = 2^36
  테이블 크기 = 2^36 × 8 = 2^39 bytes = 512 GB ← 비현실적!
```

해결: 4단계 페이지 테이블 (Intel Core i7)

```
48-bit 가상 주소:
  ┌──────┬──────┬──────┬──────┬──────────────┐
  │VPN1  │VPN2  │VPN3  │VPN4  │    VPO       │
  │9 bits│9 bits│9 bits│9 bits│   12 bits    │
  └──────┴──────┴──────┴──────┴──────────────┘

변환 과정:
  CR3 → L1 PT base address
  L1 PT[VPN1] → L2 PT base address
  L2 PT[VPN2] → L3 PT base address
  L3 PT[VPN3] → L4 PT base address
  L4 PT[VPN4] → PPN

L1 각 항목이 커버하는 범위: 2^(9+9+9+12) = 2^39 = 512GB
→ 사용하지 않는 범위의 L1 PTE = null → 하위 테이블 불필요
→ 메모리 절약!
```

Linux 프로세스 가상 주소 공간

```
높은 주소 (0xFFFFFFFFFFFFFFFF)
┌──────────────────────────┐
│       커널 공간          │ ← 모든 프로세스 공유, 유저 접근 불가
├──────────────────────────┤ 0x7FFFFFFFFFFFFFFF
│       User Stack         │ ← 지역 변수, 반환 주소 (↓ 성장)
```

```
│          (%rsp)                  │
├──────────────────────────────────┤
│    (사용하지 않는 공간)          │
├──────────────────────────────────┤
│   Memory-mapped Regions          │ ← shared libraries, mmap 파일
│   (공유 라이브러리)              │
├──────────────────────────────────┤
│       Run-time Heap              │ ← malloc/new (↑ 성장)
│         (brk)                    │
├──────────────────────────────────┤
│   .bss (미초기화 전역변수)       │
│   .data (초기화된 전역변수)      │
│   .text (실행 코드)              │
└──────────────────────────────────┘ 0x0000000000400000
```

Memory Mapping (mmap)

```
        void *mmap(void *start, int len, int prot, int flags, int fd, int
offset);
```

파일이나 익명 메모리를 프로세스의 가상 주소 공간에 매핑

```
prot: PROT_READ, PROT_WRITE, PROT_EXEC
flags: MAP_SHARED, MAP_PRIVATE, MAP_ANON

MAP_PRIVATE + COW:
  → fork()로 프로세스 복제 시, 물리 페이지를 공유
  → 어느 한쪽이 쓰기를 시도 → Protection Fault
  → 핸들러가 새 물리 페이지에 복사 (Copy-on-Write)
  → 쓰기 완료
  → 복사 비용을 실제 필요 시점으로 지연
```

Ch7. Dynamic Memory Allocation (malloc)

동적 메모리 할당의 필요성

```
정적 할당 (컴파일 시간에 크기 결정):
  int arr[100];  // 컴파일 시 크기 고정

동적 할당 (런타임에 크기 결정):
  int *arr = malloc(n * sizeof(int));  // n은 런타임에 결정
  → 사용자 입력, 파일 크기 등에 대응 가능
```

malloc API

```
        void *malloc(size_t size)
          → 최소 size bytes의 메모리 블록 포인터 반환
          → x86-64: 16-byte 정렬 보장
          → size==0: NULL 반환
          → 실패: NULL 반환, errno 설정

        void free(void *p)
```

```
  → p가 가리키는 블록을 가용 풀로 반환
  → p는 반드시 malloc/calloc/realloc이 반환한 포인터여야 함

calloc(n, size):  n*size 바이트, 0으로 초기화
realloc(p, size): 기존 블록 크기 변경
sbrk(n):          힙을 n바이트 증가 (할당기가 내부적으로 사용)
```

단편화 (Fragmentation)

내부 단편화 (Internal Fragmentation):

```
블록 크기 > 페이로드 크기인 경우 낭비되는 공간

원인:
  1. 정렬 패딩: malloc(1) 요청 → 16 bytes 블록 반환 (15 bytes 낭비)
  2. 헤더 오버헤드: 각 블록마다 8 bytes 헤더 필요
```

외부 단편화 (External Fragmentation):

```
가용 메모리 합계는 충분하지만 연속된 블록이 없는 경우

예:
  [free:50][alloc:100][free:50]
  → malloc(120) 요청 → 합계 100 bytes 가용하지만 연속 블록 없음 → 실패!

해결: Coalescing (인접 가용 블록 합병)
```

메모리 성능 지표

```
Aggregate payload Pₖ: k번째 요청 시점의 현재 할당된 페이로드 합계
Current heap size Hₖ: k번째 요청 후 힙 크기 (sbrk로만 증가, 감소 안됨)

Memory Overhead Oₖ = max(Pᵢ, i≤k) / Hₖ
                   = 최대 누적 페이로드 / 현재 힙 크기

Oₖ = 1.0이 이상적 (낭비 없음)
Oₖ < 1.0은 단편화로 인한 낭비
```

가용 블록 추적: 3가지 방법

방법 1: Implicit Free List

```
블록 구조:
  ┌──────────────────────────────────┐
  │   Header (1 word = 4 or 8 bytes) │
  │   [size (상위 비트) | alloc(최하위 비트)] │
  ├──────────────────────────────────┤
  │         Payload                  │
  ├──────────────────────────────────┤
  │    Padding (선택적, 정렬용)     │
  └──────────────────────────────────┘

트릭: size의 하위 비트는 항상 0 (정렬로 인해)
      → alloc bit를 하위 비트에 저장!

예: size=32, alloc=1 → header = 0x21 (00100001)
```

```
    size 추출: header & ~0x7 = 32
    alloc 추출: header & 0x1 = 1

힙 순회:
  current = heap_start
  while (current != heap_end):
      size = get_size(current)
      alloc = get_alloc(current)
      next = (char*)current + size

성능:
  할당: O(전체 블록 수) - 선형 탐색
  해제: O(1) - alloc bit만 clear
  현대 할당기에서는 사용하지 않음 (너무 느림)
```

방법 2: Explicit Free List

```
가용 블록에만 prev/next 포인터 저장:

가용 블록 구조:
  ┌──────────────────┐
  │   Header (size|0)    │
  ├──────────────────┤
  │   Next free pointer  │  ← 다음 가용 블록 가리킴
  ├──────────────────┤
  │   Prev free pointer  │  ← 이전 가용 블록 가리킴
  ├──────────────────┤
  │   (나머지 공간)      │
  └──────────────────┘

할당된 블록: 기존과 동일 (포인터 없음)

삽입 정책:
  LIFO: 새로 해제된 블록을 리스트 앞에 삽입 (구현 단순)
  FIFO: 리스트 끝에 삽입
  주소순: 블록 주소 순서 유지 (단편화 감소 효과)

성능:
  할당: O(가용 블록 수) - 가용 블록만 탐색
  해제: O(1) - 리스트 앞에 삽입
```

방법 3: Segregated Free List (분리 가용 리스트)

```
크기 클래스(Size Class)별로 별도의 가용 리스트 유지:

  클래스 [1-8 bytes]:    → [4] → [8] → NULL
  클래스 [9-16 bytes]:   → [16] → [12] → NULL
  클래스 [17-32 bytes]:  → [32] → [20] → NULL
  클래스 [33-∞ bytes]:   → [100] → [64] → NULL

할당 과정:
  1. 요청 크기에 맞는 클래스 탐색
  2. 해당 클래스에서 first-fit
  3. 블록 발견 시 split 후 할당
  4. 해당 클래스에 맞는 블록 없으면 더 큰 클래스로
  5. 모두 없으면 sbrk()로 힙 확장

성능:
  할당: O(1) 근사 (크기 클래스별 즉시 접근)
```

```
  → first-fit ≈ best-fit (작은 단편 분리됨)

이것이 현대 malloc(glibc, tcmalloc 등)의 핵심 아이디어
```

Splitting과 Coalescing

Splitting (분할):

```
64-byte 가용 블록에 32 bytes 요청:
  Before: [free: 64 bytes]
  After:  [alloc: 32 bytes] [free: 32 bytes]
  → 필요한 만큼만 할당, 나머지는 새 가용 블록으로
```

Coalescing (합병):

```
free(p) 시 인접 가용 블록과 합병:
  Before: [free:32] [alloc:p: 32] [free:16]
  free(p) 후 합병:
  After:  [free: 80 bytes]

False Fragmentation 방지:
  합병 없으면: [free:32] [free:32] [free:16]
  → malloc(70) 불가! (합계 80 bytes지만 연속 블록 없음)

정책:
  Immediate Coalescing: free() 즉시 합병 (코드 간단)
  Deferred Coalescing: 나중에 일괄 합병 (성능 개선 가능)
```

Garbage Collection (GC)

Reference Counting:

```
각 블록에 참조 횟수(ref count) 저장
참조 생성 시: count++
참조 소멸 시: count--
count == 0 시: 자동 해제

예시:
node_t *a = gen_node(10, NULL);  // count=1
node_t *b = gen_node(20, a);    // a의 count=2
a = b;                           // 이전 a 노드 count=1
b = ...;                         // b가 가리키던 노드 count=0 → 자동 해
제

단점: 순환 참조 처리 불가!
  A → B → A: 서로 참조하면 count는 항상 ≥ 1
  → 실제로는 도달 불가능하지만 해제 안됨

사용: Chrome/Firefox C++ (smart pointers)
```

Mark & Sweep:

```
단계 1 - Mark (마크):
  Root에서 시작하여 DFS/BFS로 도달 가능한 모든 블록을 마크
  Root: 레지스터, 스택 변수, 전역 변수에 있는 힙 포인터

단계 2 - Sweep (스윕):
  힙 전체를 순회하면서 마크되지 않은 블록을 가용 리스트에 추가
```

```
특징:
  + 순환 참조도 처리 가능 (도달 불가능하면 회수)
  - Stop-the-world: GC 중 프로그램 일시 중단
  - 예측 불가능한 지연 발생

사용: Java, Python, Go, JavaScript
```

힙 관련 보안 취약점

Heap Overflow → Arbitrary Write:

```
char *p = malloc(16);
memcpy(p, src, 32);  // 16 bytes 초과 → 다음 블록 메타데이터 오염

// K&R malloc의 free list 연결 구조:
// node->prev->next = node->next;  (합병 시 실행)

// 공격자가 next/prev 덮어쓰면:
// node->prev->next = 공격자가 원하는 주소
// → 임의 메모리 쓰기 (Arbitrary Write)
```

Use-After-Free (UAF):

```
// Chromium 실제 사례:
Doc *doc = new Doc();
Body *body = new Body();
doc->child = body;

delete body;           // body 해제 (dangling pointer!)
doc->child->getAlign();  // 해제된 메모리 접근 → UAF

// 공격 시나리오:
// 1. delete body (dangling pointer 생성)
// 2. 공격자 객체로 freed 메모리를 채움 (Heap Spray)
// 3. doc->child->getAlign() → 공격자의 fake vtable 실행
// → 제어 흐름 탈취 (Control Flow Hijack)
```

Double Free:

```
char *p = malloc(100);
free(p);
free(p);  // 동일 블록을 두 번 해제
          // 가용 리스트에 같은 블록이 두 번 등장
          // → malloc이 같은 블록을 두 곳에 할당
          // → 두 포인터가 같은 메모리 가리킴 → 데이터 오염
```

Ch8. Exceptions (예외)

Exceptional Control Flow (ECF)

일반적인 제어 흐름은 순차적 실행, 점프, 함수 호출로 이루어진다. 하지만 이것만으로는 부족한 상황들이 있다:

```
- 타이머가 만료됐을 때 OS가 CPU를 회수해야 함
- 네트워크 패킷이 도착했을 때 즉시 처리해야 함
```

```
- 0으로 나누기가 발생했을 때 처리해야 함
- 사용자가 Ctrl-C를 눌렀을 때 프로그램을 종료해야 함
```

Exception: 어떤 이벤트에 반응하여 OS 커널로 제어를 전달하는 메커니즘

```
User Code 실행 중 이벤트 발생
          ↓
  커널로 제어 전달 (Exception Table 참조)
          ↓
  Exception Handler 실행
          ↓
  ① I_next로 복귀 (Interrupt, Trap)
  ② I_current 재실행 (Fault - 복구 가능)
  ③ 프로그램 종료 (Fault - 복구 불가, Abort)
```

Exception Table (예외 테이블)

```
OS가 프로그램 로드 시 구성하는 테이블:
  인덱스 k → 핸들러 주소

Exception Table:
  [0] → handler_0 주소
  [1] → handler_1 주소
  ...
  [14] → Page Fault 핸들러 주소
  [13] → General Protection Fault 핸들러
  [n-1] → handler_n-1 주소

이벤트 발생 시 CPU는:
  1. 이벤트 번호 k 결정
  2. exception_table[k]의 핸들러로 점프
```

예외의 4가지 분류

Interrupt (인터럽트) — 비동기

```
발생 원인: CPU 외부 장치 (타이머, 키보드, NIC, 디스크)
발생 방식: 비동기 (명령어 실행과 무관하게 발생)
반환 위치: I_next (현재 명령어 다음 명령어로 복귀)

예시:
  타이머 인터럽트: 매 수 ms마다 발생
    → OS가 유저 프로그램에서 CPU를 회수 (Kernel Preemption)
    → 다른 프로세스로 Context Switch 가능
  키보드 Ctrl-C: SIGINT 발생
  NIC 패킷 도착: 네트워크 데이터 처리
```

Trap (트랩) — 동기, 의도적

```
발생 원인: 명령어 실행의 결과 (의도적)
발생 방식: 동기 (syscall 명령어 실행 시)
반환 위치: I_next (syscall 다음 명령어로 복귀)

예시:
  System Call: open(), read(), fork() 등
  GDB Breakpoint: int 3 (0xcc) 명령어 실행
```

```
System Call 흐름:
  유저 코드에서 open("file.txt", O_RDONLY) 호출
       ↓
  libc의 __open() 함수:
    %eax = 2        # open syscall 번호
    syscall         # Trap 발생 → 커널로 전환
       ↓
  커널: syscall_table[2] = sys_open() 실행
  파일 열기 수행
       ↓
  반환값 → %rax
  유저 모드로 복귀 → __open()의 다음 명령어 실행
```

Fault (폴트) — 동기, 비의도적, 복구 가능

```
발생 원인: 명령어 실행의 결과 (비의도적)
발생 방식: 동기
반환 위치: I_current (복구 후 재실행) 또는 프로그램 종료

예시:
  Page Fault: 접근한 페이지가 DRAM에 없음
    → 복구 가능: 디스크에서 페이지 로드 후 재실행
    → 복구 불가: 유효하지 않은 주소 → SIGSEGV
  Protection Fault: 권한 없는 메모리 접근
    → 복구 불가 → SIGSEGV
```

Abort (어보트) — 동기, 비의도적, 복구 불가

```
발생 원인: 심각한 오류
발생 방식: 동기
반환 위치: 프로그램 즉시 종료

예시:
  불법 명령어 실행 (SIGILL)
  패리티 오류 (하드웨어 오류)
  Machine Check Exception
```

System Call 상세

```
System Call 번호 할당 (x86-64 Linux):
  0: read     1: write    2: open    3: close
  57: fork   59: execve  60: _exit  62: kill

Convention:
  %rax: syscall 번호
  %rdi: 1번째 인자
  %rsi: 2번째 인자
  %rdx: 3번째 인자
  %r10: 4번째 인자
  %r8:  5번째 인자
  %r9:  6번째 인자
  반환값: %rax (음수이면 오류)

실제 어셈블리 (open() syscall):
  mov $0x2, %eax    # open = syscall #2
  syscall           # Trap → 커널 모드
  cmp $0xfffffffffffff001, %rax  # 오류 확인
```

Ch9. Processes (프로세스)

프로세스의 두 가지 핵심 추상화

1. Logical Control Flow (논리적 제어 흐름)

```
프로세스는 CPU를 독점하는 것처럼 보인다.

실제로는 여러 프로세스가 시분할로 CPU 공유:
Process A: ████████▒▒▒
```

동작:

```
1. 현재 프로세스의 완전한 복사본 생성
   - 가상 주소 공간 (페이지 테이블 복사 + COW 설정)
   - 파일 디스크립터 테이블
   - 시그널 핸들러 등

2. 두 번 반환:
   - 부모: 자식의 PID (양수) 반환
   - 자식: 0 반환

3. 실행 순서: 비결정적 (OS 스케줄러가 결정)
```

COW (Copy-on-Write) 최적화:

```
fork() 직후: 부모와 자식이 동일한 물리 페이지 공유
             모든 페이지를 Read-Only로 표시

어느 한쪽이 쓰기 시도:
  → Protection Fault 발생
  → 커널이 해당 페이지를 새로운 물리 페이지에 복사
  → 쓰기 완료

이점: 대부분의 페이지를 복사하지 않아도 됨 (execve 후 어차피 교체)
     fork() 비용이 극적으로 감소
```

fork() 예시:

```c
int main() {
    pid_t pid;
    int x = 1;

    pid = fork();

    if (pid == 0) {    // 자식
        printf("child: x=%d\n", ++x);  // x=2 출력
        return 0;
    } else {           // 부모
        printf("parent: x=%d\n", --x); // x=0 출력
        return 0;
    }
}
// 출력 순서: 비결정적 (child 먼저 or parent 먼저)
// x 값: 완전히 독립적 (자식의 x 변경이 부모의 x에 영향 없음)
```

Process Graph (프로세스 그래프) 분석

```c
void fork2() {
    printf("L0\n");
    fork();
    printf("L1\n");
    fork();
    printf("Bye\n");
}
```

```
프로세스 그래프:
  main: printf(L0) → fork() ──────────────────────────────────────→
printf(Bye)
                                  |                                  ↑
                      Parent: printf(L1) → fork() ──────────────────|
```

```
                              |           |                     |
                    Child1:   |         printf(Bye)
printf(L1)
                              |                                 |
                    ...     Child2: printf(L1) → fork() →
printf(Bye)
                                                    |
                                              Child3: printf(Bye)

총 출력: L0×1, L1×2, Bye×4

가능한 순서 (Topological Sort):
  L0 → L1 → Bye → L1 → Bye → Bye → Bye  (OK)
  L0 → L1 → L1 → Bye → Bye → Bye → Bye  (OK)

불가능한 순서:
  L1 → L0 → ...  (L0이 항상 fork 전이므로 불가)
```

Zombie와 Orphan

Zombie Process:

```
자식이 exit() 호출로 종료 → 모든 자원 해제
하지만 exit status는 보존 필요 (부모가 wait() 호출할 때까지)
→ 일부 자원 유지 = Zombie 상태

ps 출력: PID TTY TIME CMD
         6640 pts/0 00:00:00 forks <defunct>  ← Zombie

부모가 wait() 호출 → exit status 회수 → Zombie 완전 해제

부모가 wait() 하지 않고 계속 실행:
  → Zombie가 누적
  → 커널의 프로세스 테이블 고갈 → 시스템 장애
```

Orphan Process:

```
부모가 자식보다 먼저 종료된 경우
→ init 프로세스 (PID=1)가 자동으로 입양
→ init이 주기적으로 wait() 호출하여 reaping
```

execve() — 새 프로그램 실행

```
int execve(char *filename, char *argv[], char *envp[])
```

동작:

```
1. 새 프로그램(filename)을 로드
2. 현재 프로세스의 코드/데이터/스택을 교체
3. PID 유지, 열린 파일 디스크립터 유지
4. 성공 시 절대 반환하지 않음 (실패 시에만 -1 반환)

메모리 재구성:
  페이지 테이블 초기화
  .text, .data, .bss를 새 프로그램으로 설정
  스택, 힙 초기화
  %rip → 새 프로그램의 entry point (_start)
```

Shell 구현 패턴 (fork + execve):

```c
// 쉘의 eval() 함수
void eval(char *cmdline) {
    char *argv[MAXARGS];
    int bg = parseline(cmdline, argv);

    if (!builtin_command(argv)) {
        if ((pid = fork()) == 0) {  // 자식
            execve(argv[0], argv, environ);  // 새 프로그램 실행
            printf("%s: Command not found\n", argv[0]);
            exit(0);
        }
        if (!bg) {
            waitpid(pid, &status, 0);  // 포그라운드: 완료 대기
        } else {
            printf("%d %s", pid, cmdline);  // 백그라운드: 즉시 반환
        }
    }
}
```

Ch10. Signals (시그널)

시그널이란?

```
예외(Exception): 하드웨어 이벤트 → 커널이 처리
시그널(Signal):  소프트웨어 이벤트 → 유저 프로세스가 처리

시그널: 커널이 프로세스에게 보내는 작은 메시지
  → 어떤 이벤트가 발생했음을 알림
  → 1~30번 정수 ID로 식별
  → 정보: ID + "발생했다"는 사실뿐 (추가 데이터 없음)
  → 큐 없음: 같은 타입 최대 1개만 pending
```

주요 시그널:

ID	이름	기본 동작	발생 원인	재정의 가능
2	SIGINT	종료	Ctrl-C	✓
9	SIGKILL	종료	강제 종료	✗ (절대 불가)
11	SIGSEGV	종료	Segmentation Fault	✓
14	SIGALRM	종료	타이머 만료	✓
17	SIGCHLD	무시	자식 종료/정지	✓
20	SIGTSTP	정지	Ctrl-Z	✓

시그널의 3가지 상태

```
커널은 각 프로세스에 대해 두 개의 비트벡터 유지:

pending 비트벡터:  전송됐지만 아직 수신되지 않은 시그널
blocked 비트벡터:  수신을 억제한 시그널 (sigprocmask로 설정)

pnb = pending & ~blocked
  → 실제로 처리할 시그널 집합

pnb의 가장 낮은 번호의 비트부터 처리
```

시그널 전송 방법

```
1. /bin/kill 명령어:
   /bin/kill -9 24818      # PID 24818에 SIGKILL
   /bin/kill -9 -24817     # 프로세스 그룹 24817 전체에 SIGKILL

2. 키보드:
   Ctrl-C → SIGINT → 포그라운드 프로세스 그룹 전체
   Ctrl-Z → SIGTSTP → 포그라운드 프로세스 그룹 전체

3. kill() 함수:
   kill(pid, signum)    # 특정 프로세스에 시그널 전송
   kill(-pgid, signum)  # 프로세스 그룹 전체에 전송

4. 커널 자동 전송:
   Page Fault → SIGSEGV
   Divide by zero → SIGFPE
   자식 종료 → 부모에게 SIGCHLD
```

시그널 수신과 처리

```
프로세스가 다음 명령어를 실행하기 직전 (컨텍스트 전환 시):
  pnb = pending & ~blocked 계산
  pnb ≠ 0이면:
    가장 작은 번호 k의 시그널부터 처리:
      기본 동작: 종료/정지/무시
      커스텀 핸들러: handler_func() 호출
    모든 pnb의 비트 처리 후
  다음 명령어로 점프
```

시그널 핸들러 설치:

```c
// 함수 포인터로 핸들러 설치
signal(SIGINT, my_handler);   // SIGINT 수신 시 my_handler 호출
signal(SIGINT, SIG_IGN);      // SIGINT 무시
signal(SIGINT, SIG_DFL);      // 기본 동작으로 복귀

void my_handler(int sig) {
    printf("Caught signal %d\n", sig);
    exit(0);
}
```

Implicit Blocking (암묵적 차단)

```
핸들러가 실행 중인 동안, 같은 타입의 시그널은 자동으로 차단된다.
(pending bit는 set되지만 delivered되지 않음)

예: SIGINT 핸들러 실행 중 Ctrl-C 두 번 누르면:
  → 첫 Ctrl-C: 핸들러 시작 + SIGINT 자동 차단
  → 두 번째 Ctrl-C: pending bit set (큐 없으므로 1개만 pending)
  → 세 번째 Ctrl-C: 이미 pending이므로 무시
  → 핸들러 종료: SIGINT 차단 해제 → 두 번째 Ctrl-C 처리

중요: 같은 타입은 큐가 없으므로 여러 번 눌러도 1번만 처리됨
```

Shell의 Zombie 문제와 SIGCHLD 해결

문제:

```c
void eval(char *cmdline) {
    // ...
    if (!bg) {
        waitpid(pid, &status, 0);  // 포그라운드: OK
    } else {
        printf("%d %s", pid, cmdline);  // 백그라운드: wait 안 함!
    }
}
// 백그라운드 자식이 종료되면 → Zombie!
// 쉘은 계속 실행 중 → 누가 wait()를 호출하나?
```

해결: SIGCHLD 핸들러:

```c
void sigchld_handler(int sig) {
    int status;
    pid_t pid;

    // ★ 반드시 while 루프 사용!
    while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
        // zombie 회수 완료
        printf("reaped child %d\n", pid);
    }
    // WNOHANG: 이미 종료된 자식만 처리, 없으면 즉시 반환
    // pid=-1: 임의의 자식 프로세스
}

// 메인에서 핸들러 등록
signal(SIGCHLD, sigchld_handler);
```

왜 while 루프인가?:

```
여러 자식이 "동시에" 종료되면:
  → SIGCHLD는 1개만 pending (큐 없음)
  → 핸들러가 1번만 실행됨
  → 1번의 waitpid만 호출하면 나머지 zombie 미회수

while 루프로 WNOHANG과 함께:
  → 한 번의 핸들러 호출에서 모든 zombie 회수 가능
```

안전한 시그널 핸들러 작성 지침

G0: 핸들러를 최대한 단순하게

```c
// 좋은 패턴: 전역 플래그만 설정
volatile sig_atomic_t child_terminated = 0;
void sigchld_handler(int sig) {
    child_terminated = 1;
}
```

G1: async-signal-safe 함수만 호출

```
async-signal-safe: write, kill, _exit, waitpid, sleep
async-signal-safe 아님: printf, sprintf, malloc

printf가 위험한 이유:
  printf는 내부적으로 락(lock)을 사용
  메인 코드가 printf 실행 중 → 핸들러가 printf 호출
  → 같은 락을 다시 요청 → Deadlock!
```

```
안전한 출력: write(STDOUT_FILENO, msg, strlen(msg))
```

G2: errno 저장/복원

```
void handler(int sig) {
    int saved_errno = errno;  // 저장
    // ... 처리 ...
    errno = saved_errno;      // 복원
}
```

G4: 전역 변수는 volatile로 선언

```
volatile long flag = 0;
// volatile 없으면 컴파일러가 레지스터에 캐시
// → 핸들러가 설정한 값을 메인이 못 볼 수 있음
```

G5: 플래그 변수는 volatile sig_atomic_t로

```
volatile sig_atomic_t flag = 0;
// sig_atomic_t: 원자적 읽기/쓰기 보장
// 이 타입의 변수에 대한 단순 대입은 동기화 불필요
```

시스템 프로그래밍 수업정리노트 (Part 3: Ch11~Ch15)

Ch11. Threads (스레드)

동시성 프로그래밍의 어려움

```
인간의 사고는 본질적으로 순차적이다.
수십 개의 스레드가 동시에 실행될 때 발생하는 모든 경우의 수를
머릿속으로 추적하는 것은 불가능에 가깝다.

대표적인 동시성 버그:
  Race Condition: 결과가 실행 순서에 따라 달라짐
  Deadlock: 서로가 상대방의 자원을 기다림
  Starvation: 특정 작업이 무한정 진행되지 못함
```

프로세스 vs 스레드

프로세스 (Process):

```
= process context + code/data/stack + kernel data structures

독립적인 가상 주소 공간:
  각 프로세스는 자신만의 page table
  fork()로 생성 (~20K cycles)

통신: IPC (파이프, 소켓, 공유 메모리)
격리: 완전 격리 → 안전하지만 느린 통신
```

스레드 (Thread):

```
= thread context + 공유 code/data/stack + 공유 kernel data structures

같은 프로세스 내 여러 제어 흐름:
  pthread_create()로 생성 (~10K cycles)

각 스레드만의 전용:
  Thread ID (TID)
  레지스터 (PC, 스택 포인터, general purpose)
  스택 (하지만 VM으로 보호되지 않음!)

모든 스레드가 공유:
  코드(.text), 초기화된 데이터(.data), 힙
  공유 라이브러리
  열린 파일 디스크립터
```

스레드 메모리 모델 — 위험성

```
스레드 스택은 VM으로 보호되지 않는다!

Thread 1 stack: 0x7fff_0000 ~ 0x7fff_ffff
Thread 2 stack: 0x7ffe_0000 ~ 0x7ffe_ffff

Thread 1이 Thread 2의 스택 주소를 알면:
  int *p = (int *)thread2_stack_address;
  *p = 42;  // Thread 2의 스택 변수를 직접 수정!
            // 아무런 오류 없이 실행됨

→ 스레드 프로그래밍에서 포인터 버그가 심각한 이유
```

Pthreads API

```c
#include <pthread.h>

// 스레드 생성
int pthread_create(pthread_t *tid,          // TID 저장 위치
                   pthread_attr_t *attr,     // 속성 (보통 NULL)
                   void *(*func)(void *),    // 스레드 함수
                   void *arg);               // 함수 인자

// 스레드 종료 대기 + 자원 회수
int pthread_join(pthread_t tid, void **retval);

// 현재 스레드 TID 반환
pthread_t pthread_self(void);

// 현재 스레드 종료
void pthread_exit(void *retval);
```

기본 예제:

```c
#include "csapp.h"

void *thread(void *vargp) {
    printf("Hello, world!\n");
    return NULL;
}

int main() {
```

```
    pthread_t tid;
    pthread_create(&tid, NULL, thread, NULL);
    pthread_join(tid, NULL);  // thread 종료 대기
    return 0;
}

// 실행 흐름:
// main: pthread_create() → peer thread 생성
// main: pthread_join() → peer가 끝날 때까지 대기
// peer: printf() → return NULL (종료)
// main: pthread_join() 반환 → exit()
```

스레드 인자 전달 — 3가지 패턴

패턴 1: malloc으로 개별 할당 (권장)

```
int hist[N] = {0};

int main() {
    pthread_t tids[N];
    for (long i = 0; i < N; i++) {
        long *p = malloc(sizeof(long));  // ← 각 스레드마다 별도 메모리
        *p = i;
        pthread_create(&tids[i], NULL, thread, (void *)p);
    }
    // ...
}

void *thread(void *vargp) {
    hist[*(long *)vargp] += 1;
    free(vargp);  // ← 스레드가 직접 해제 (Producer-Consumer 패턴)
    return NULL;
}

장점: 각 스레드가 독립적인 인자 복사본 보유
주의: 스레드 내에서 반드시 free() 호출
```

패턴 2: 정수 캐스팅 (간단)

```
for (long i = 0; i < N; i++) {
    pthread_create(&tids[i], NULL, thread, (void *)i);
    //                                              ↑ 포인터가 아닌 값
자체
}

void *thread(void *vargp) {
    hist[(long)vargp] += 1;  // 포인터를 다시 정수로 캐스팅
    return NULL;
}

가능 조건: sizeof(long) <= sizeof(void *)
           (64-bit 시스템에서는 항상 성립)
```

패턴 3: 루프 변수 주소 전달 (WRONG!)

```
for (int i = 0; i < N; i++) {
    pthread_create(&tids[i], NULL, thread, (void *)&i);
    //                                              ↑ 루프 변수 i의 주
소
}
```

```
void *thread(void *vargp) {
    hist[*(int*)vargp] += 1;  // *vargp = i의 현재 값
    return NULL;
}

문제: 모든 스레드가 동일한 &i를 가리킴
     스레드 실행 전에 i가 이미 증가해 있을 수 있음
     → Race Condition!
```

공유 변수 분석

```
char **ptr;  // 전역 변수

int main() {
    long i;
    pthread_t tid;
    char *msgs[2] = {"Hello from foo", "Hello from bar"};

    ptr = msgs;  // 전역 포인터 설정
    for (i = 0; i < 2; i++)
        pthread_create(&tid, NULL, thread, (void *)i);
    pthread_exit(NULL);
}

void *thread(void *vargp) {
    long myid = (long)vargp;
    static int cnt = 0;
    printf("[%ld]: %s (cnt=%d)\n", myid, ptr[myid], ++cnt);
    return NULL;
}
```

변수별 공유 여부:

변수	인스턴스	메인?	스레드0?	스레드1?	공유?
ptr (global)	1개	YES	YES	YES	YES ★
cnt (static)	1개	NO	YES	YES	YES ★
i (main 로컬)	1개	YES	NO	NO	NO
msgs (main)	1개	YES	YES	YES	YES ★
myid (로컬)	2개	NO	YES(0)	YES(1)	NO (각자 독립)

규칙: **변수 x는 여러 스레드가 x의 어떤 인스턴스를 참조하면 공유됨**

Ch12. Synchronization (동기화)

Race Condition (레이스 컨디션) — 원인 분석

```
volatile long cnt = 0;
void *thread(void *vargp) {
    for (long i = 0; i < niters; i++)
        cnt++;  // ← 이 줄이 문제!
    return NULL;
}
```

cnt++의 어셈블리 분해:

```
movq cnt(%rip), %rdx    # L: Load — cnt 값을 %rdx에 로드
addq $1, %rdx            # U: Update — %rdx += 1
movq %rdx, cnt(%rip)    # S: Store — %rdx 값을 cnt에 저장
```

잘못된 인터리빙 예시 (cnt=0에서 시작):

```
Thread 1          Thread 2          cnt    %rdx1  %rdx2
L1 (rdx1=0)                          0      0      -
U1 (rdx1=1)                          0      1      -
              L2 (rdx2=0)            0      1      0
              U2 (rdx2=1)            0      1      1
S1 (cnt=1)                           1      1      1
              S2 (cnt=1)             1      1      1
결과: cnt=1 (기대값: 2) → Race Condition!
```

Progress Graph (진행 그래프)

```
2D 그래프로 두 스레드의 실행 상태 표현:

          H1  L1  U1  S1  T1
     H2   .   .   .   .   .
     L2   .   .   .   .   .
     U2   .   .   .   .   .
     S2   .   .   .   .   .
     T2   .   .   .   .   .

각 점 (a, b) = Thread 1이 a를 완료, Thread 2가 b를 완료한 상태

Unsafe Region: L1-U1-S1과 L2-U2-S2가 겹치는 영역
→ 이 영역을 피하는 trajectory = Safe trajectory = 올바른 실행
```

Mutual Exclusion (상호 배제)

Mutex (뮤텍스)

개념:

```
Critical Section: 공유 자원에 접근하는 코드 구간
Mutex: Critical Section에 대한 상호 배제 보장
       → 한 번에 하나의 스레드만 Critical Section 진입 가능
```

구현 원리 (test_and_set):

```
// test_and_set은 원자적(atomic) 명령어로 구현됨
int test_and_set(int *ptr) {
    int old = *ptr;      // ← 이 3줄이
    *ptr = LOCKED;       //     원자적으로
    return old;          //     실행됨 (중단 불가)
}

void lock(mutex_t *m) {
    while (test_and_set(&m->state) == LOCKED)
        ;  // spinning
}

void unlock(mutex_t *m) {
    m->state = UNLOCKED;
}
```

pthreads Mutex 사용:

```
pthread_mutex_t mutex;
pthread_mutex_init(&mutex, NULL);

void *thread(void *vargp) {
    for (long i = 0; i < niters; i++) {
        pthread_mutex_lock(&mutex);
        cnt++;  // Critical Section
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

// 결과: 항상 올바른 cnt 값 (하지만 ~18배 느림)
// 성능 비교:
//   Mutex 없음: 12.0 ms
//   Mutex 있음: 214.0 ms (약 18배 느림)
```

Semaphore (세마포어)

```
세마포어: 음수가 될 수 없는 정수형 동기화 변수

P 연산 (wait):
  while (s == 0) wait();  // s가 양수가 될 때까지 대기
  s--;                     // 원자적으로 감소
  → "자원을 획득한다"

V 연산 (post):
  s++;  // 원자적으로 증가
  // 대기 중인 스레드 하나를 깨움
  → "자원을 반납한다"

불변: s >= 0 항상 유지!
```

pthreads Semaphore 사용:

```
#include <semaphore.h>

sem_t s;
sem_init(&s, 0, 1);   // s = 1 (초기값)
sem_wait(&s);          // P 연산
sem_post(&s);          // V 연산
```

Producer-Consumer 패턴

1-element Buffer

```
struct {
    int buf;
    sem_t full;    // 초기값 0: "버퍼에 데이터 없음"
    sem_t empty;   // 초기값 1: "버퍼에 빈 공간 있음"
} shared;

void *producer(void *arg) {
    for (int i = 0; i < NITERS; i++) {
        int item = produce();
        sem_wait(&shared.empty);   // 빈 슬롯 기다림 (empty: 1→0)
```

```c
        shared.buf = item;          // 생산 (쓰기)
        sem_post(&shared.full);     // "데이터 있음" 알림 (full: 0→1)
    }
}

void *consumer(void *arg) {
    for (int i = 0; i < NITERS; i++) {
        sem_wait(&shared.full);     // 데이터 기다림 (full: 1→0)
        int item = shared.buf;      // 소비 (읽기)
        sem_post(&shared.empty);    // "빈 슬롯 있음" 알림 (empty: 0→1)
        consume(item);
    }
}
```

실행 추적:

```
초기: empty=1, full=0, buf=?

Producer sem_wait(empty): empty=1→0 (통과)
Producer buf=item:         buf=42
Producer sem_post(full):   full=0→1

Consumer sem_wait(full):   full=1→0 (통과)
Consumer item=buf:         item=42
Consumer sem_post(empty):  empty=0→1

Producer sem_wait(empty):  empty=1→0 (통과)
...
```

n-element Buffer (Bounded Buffer)

```c
typedef struct {
    int *buf;               // 원형 배열
    int n;                  // 버퍼 크기
    int front;              // 다음에 꺼낼 위치
    int rear;               // 다음에 넣을 위치
    pthread_mutex_t mutex;  // 버퍼 접근 보호
    sem_t empty_slots;      // 초기값 = n
    sem_t full_slots;       // 초기값 = 0
} sbuf_t;

void sbuf_insert(sbuf_t *sp, int item) {
    sem_wait(&sp->empty_slots);        // 빈 슬롯 대기
    pthread_mutex_lock(&sp->mutex);    // 버퍼 잠금
    sp->buf[++sp->rear % sp->n] = item;  // 삽입
    pthread_mutex_unlock(&sp->mutex);  // 버퍼 해제
    sem_post(&sp->full_slots);         // 소비자에게 알림
}

int sbuf_remove(sbuf_t *sp) {
    int item;
    sem_wait(&sp->full_slots);         // 데이터 대기
    pthread_mutex_lock(&sp->mutex);    // 버퍼 잠금
    item = sp->buf[++sp->front % sp->n];  // 제거
    pthread_mutex_unlock(&sp->mutex);  // 버퍼 해제
    sem_post(&sp->empty_slots);        // 생산자에게 알림
    return item;
}

// 핵심: mutex와 세마포어의 순서!
```

```
        // ★ 반드시: sem_wait → lock → ... → unlock → sem_post
        // 반대 순서(lock → sem_wait)면 Deadlock 발생 가능!
```

Readers-Writers Problem

```
상황:
  Reader: 공유 객체를 읽기만 함
  Writer: 공유 객체를 수정 (읽기/쓰기)

규칙:
  여러 Reader 동시 접근: 가능 (충돌 없음)
  Writer는 단독 접근 필요 (다른 Reader/Writer 불가)

해결 (First Readers-Writers Problem - 독자 우선):
```

```
        int readcnt = 0;
        sem_t mutex;  // readcnt 보호 (초기값 1)
        sem_t w;      // 쓰기 잠금 (초기값 1)

        void reader(void) {
            while (1) {
                sem_wait(&mutex);
                readcnt++;
                if (readcnt == 1)      // 첫 번째 독자
                    sem_wait(&w);      // Writer 차단
                sem_post(&mutex);

                /* 읽기 수행 */

                sem_wait(&mutex);
                readcnt--;
                if (readcnt == 0)      // 마지막 독자
                    sem_post(&w);      // Writer 허용
                sem_post(&mutex);
            }
        }

        void writer(void) {
            while (1) {
                sem_wait(&w);
                /* 쓰기 수행 */
                sem_post(&w);
            }
        }
```

순서 분석 (R1 R2 W1 R3 도착):

```
R1 도착: readcnt=1, w=0 (Writer 차단), R1 읽기 시작
R2 도착: readcnt=2, w=0 유지 (이미 차단됨), R2 읽기 시작
W1 도착: sem_wait(w) → w=0이므로 대기
R3 도착: readcnt=3, R3 읽기 시작 (W1보다 R3 우선!)
R1 종료: readcnt=2
R2 종료: readcnt=1
R3 종료: readcnt=0, sem_post(w) → W1 깨어남
W1 실행: 쓰기 수행
```

Deadlock (교착 상태)

Deadlock 4가지 필요 조건:

```
1. Mutual Exclusion (상호 배제):
   자원을 한 번에 하나의 프로세스만 사용

2. Hold and Wait (점유 대기):
   자원을 보유하면서 다른 자원 대기

3. No Preemption (비선점):
   자원은 자발적으로만 반납 가능

4. Circular Waiting (순환 대기):
   A→B→C→A 형태의 순환 자원 대기

→ 4가지 조건이 모두 성립해야 Deadlock 발생
→ 하나라도 깨면 Deadlock 방지 가능!
```

Deadlock 예시:

```c
void *count(void *vargp) {
    int id = (int)vargp;
    for (int i = 0; i < NITERS; i++) {
        sem_wait(&mutex[id]);      // Thread 0: sem_wait(mutex[0])
        sem_wait(&mutex[1-id]);    // Thread 0: sem_wait(mutex[1])
        cnt++;
        sem_post(&mutex[id]);
        sem_post(&mutex[1-id]);
    }
}
// Thread 0: wait(s0) → wait(s1) → ...
// Thread 1: wait(s1) → wait(s0) → ...

// 가능한 시나리오:
// T0: wait(s0) 성공 (s0=0)
// T1: wait(s1) 성공 (s1=0)
// T0: wait(s1) → s1=0 → 영원히 대기
// T1: wait(s0) → s0=0 → 영원히 대기
// → Deadlock!
```

Deadlock 회피 — 동일한 순서로 락 획득:

```c
void *count(void *vargp) {
    for (int i = 0; i < NITERS; i++) {
        sem_wait(&mutex[0]);   // Thread 0: s0 먼저
        sem_wait(&mutex[1]);   // Thread 0: s1 두 번째
        cnt++;
        sem_post(&mutex[id]);
        sem_post(&mutex[1-id]);
    }
}
// Thread 0: wait(s0) → wait(s1) → ...
// Thread 1: wait(s0) → wait(s1) → ...  ← 동일 순서!

// Circular Waiting 조건 제거 → Deadlock 불가
```

Thread Safety (스레드 안전성)

Class 1: 공유 변수를 보호하지 않는 함수

```c
// 위험한 예:
```

```c
static long cnt = 0;
long increment(void) { return ++cnt; }  // 보호 없음!

// 해결: mutex로 cnt 보호
```

Class 2: 여러 호출에 걸쳐 상태를 유지하는 함수

```c
// 위험한 예: rand()
static unsigned int next = 1;  // 전역 static 상태!
int rand(void) {
    next = next * 1103515245 + 12345;
    return (unsigned int)(next/65536) % 32768;
}

// 해결: 상태를 인자로 전달
int rand_r(unsigned int *nextp) {
    *nextp = (*nextp) * 1103515245 + 12345;
    return (unsigned int)(*nextp/65536) % 32768;
}
// → 각 스레드가 자신만의 nextp를 관리
```

Class 3: 스레드 안전하지 않은 함수를 호출하는 함수

```
Class 1, 2 함수를 호출하는 함수도 thread-unsafe
해결: 호출하는 함수 자체를 thread-safe하게 만들거나
     thread-safe 버전(rand_r 등)으로 교체
```

Thread/Signal 상호작용과 Deadlock

```
라이브러리 함수는 내부적으로 락을 사용한다:
  printf → fprintf → 내부 버퍼 접근 시 락 사용
  malloc → 내부 가용 리스트 접근 시 락 사용

위험 시나리오:
  메인 코드: printf(...) 실행 시작
             → 내부 락 획득
  시그널 발생 → 핸들러 실행
  핸들러: printf(...) 호출
          → 이미 획득된 락을 다시 요청
          → Deadlock!

규칙: 시그널 핸들러 내에서 printf, malloc 등 사용 금지
      → async-signal-safe 함수만 사용 (write, kill, _exit 등)
```

Memory Consistency Model

```
CPU는 성능을 위해 메모리 연산 순서를 재정렬할 수 있다.

Sequential Consistency (순차 일관성):
  모든 연산이 하나의 전역 순서로 실행되는 것처럼 보임
  가장 직관적이지만 구현 시 성능 저하 심각

Total Store Ordering (TSO) — x86:
  Store Buffer 허용: Store 연산이 Load보다 뒤에 보일 수 있음
  x86은 TSO 모델 사용

Weak Ordering — ARM:
  거의 모든 종류의 재정렬 허용
```

```
  메모리 barrier 명령어로 순서 강제

예시 (Initially A=B=0):
Thread 1: A=1; if(B==0) print "Hello";
Thread 2: B=1; if(A==0) print "World";

SC에서: "Hello"와 "World" 중 최대 하나만 출력
TSO에서: "Hello World" 또는 "World Hello" 모두 가능
         (store buffer로 인해 A=1, B=1이 늦게 다른 코어에 반영될 수 있음)
```

Ch13. System-Level I/O

Unix 철학: 모든 것은 파일이다

```
Linux에서 모든 것이 파일 인터페이스로 추상화됨:
  /dev/sda2    : 디스크 파티션
  /dev/tty2    : 터미널
  /proc/       : 커널 데이터 구조에 대한 가상 파일시스템
  /dev/null    : 항상 빈 파일
  소켓         : 네트워크 연결

이 덕분에 open, read, write, close 4개 함수로 모든 I/O 처리 가능
```

Unix I/O 기본 함수

```
// 파일 열기
int fd = open("/etc/hosts", O_RDONLY);  // 반환: fd (>=0) or -1
// O_RDONLY, O_WRONLY, O_RDWR
// O_CREAT, O_TRUNC, O_APPEND

// 파일 닫기
int ret = close(fd);
// ★ 닫힌 fd를 다시 close하면 double-close (double-free와 유사)
//   → close 후 fd = -1로 설정하는 습관 필요

// 파일 읽기
ssize_t nbytes = read(fd, buf, sizeof(buf));
// 반환: 읽은 바이트 수 (0=EOF, -1=오류)
// nbytes < sizeof(buf) → Short Count (오류 아님!)

// 파일 쓰기
ssize_t nbytes = write(fd, buf, sizeof(buf));
// 반환: 쓴 바이트 수 (-1=오류)
// nbytes < sizeof(buf) → Short Count

// 파일 위치 변경
off_t pos = lseek(fd, offset, SEEK_SET);
// SEEK_SET: 파일 시작부터
// SEEK_CUR: 현재 위치부터
// SEEK_END: 파일 끝부터
```

Short Count (짧은 읽기/쓰기)

```
Short Count: read/write가 요청한 바이트보다 적게 처리하는 현상

발생 상황:
  1. EOF 도달: 남은 바이트 < 요청 바이트
  2. 터미널에서 읽기: Enter 키 전까지 입력된 양
  3. 네트워크 소켓: 패킷이 도착한 양

★ Short Count는 오류가 아니다!

올바른 처리 패턴:
  // 정확히 n 바이트를 읽을 때까지 반복
  ssize_t rio_readn(int fd, void *buf, size_t n) {
      size_t nleft = n;
      ssize_t nread;
      char *bufp = (char *)buf;
      while (nleft > 0) {
          if ((nread = read(fd, bufp, nleft)) < 0) {
              if (errno == EINTR)  // 시그널에 의해 중단
                  nread = 0;
              else
                  return -1;  // 실제 오류
          } else if (nread == 0) {
              break;  // EOF
          }
          nleft -= nread;
          bufp += nread;
      }
      return (n - nleft);
  }
```

커널의 파일 표현: 3개의 테이블

```
Descriptor Table         Open File Table        v-node Table
[프로세스별 존재]          [전체 공유]              [전체 공유]

fd 0 ──────────────→ pos=0,refcnt=1 ───→ FILE TYPE: stdin
fd 1 ──────────────→ pos=0,refcnt=1 ───→ FILE TYPE: stdout
fd 2 ──────────────→ pos=0,refcnt=1 ───→ FILE TYPE: stderr
fd 3 ──────────────→ pos=1024,refcnt=1 ─→ FILE: disk.txt
                                              size: 4096
                                              type: REG

Open File Table 항목:
  file pos: 현재 읽기/쓰기 위치
  refcnt: 이 항목을 가리키는 fd 수
  → refcnt=0이 되면 항목 해제

v-node Table 항목:
  stat 구조체 내용 (파일 크기, 타입, 권한 등)
```

fork() 후 파일 공유

```
fork() 전:
  부모 fd 테이블: fd1→OFT_A, fd4→OFT_B
  OFT_A: pos=0, refcnt=1
  OFT_B: pos=100, refcnt=1

fork() 후:
```

```
  부모 fd 테이블: fd1→OFT_A, fd4→OFT_B
  자식 fd 테이블: fd1→OFT_A, fd4→OFT_B  ← 동일 OFT 항목 공유!
  OFT_A: pos=0, refcnt=2  ← refcnt 증가
  OFT_B: pos=100, refcnt=2

중요한 결과:
  부모와 자식이 동일한 file pos 공유
  부모가 read() 하면 pos 변경 → 자식에게도 영향
  (이것이 의도적인 동작인지 확인 필요)

close() 시: refcnt-- → 0이 되면 OFT 항목 해제
```

I/O Redirection (I/O 리다이렉션)

`ls > foo.txt` 구현 방법:

```
        // dup2(oldfd, newfd): newfd를 oldfd의 복사본으로 만듦
        // newfd가 열려있으면 먼저 닫음

        // 구현:
        int fd = open("foo.txt", O_WRONLY | O_CREAT | O_TRUNC, 0666);  //
fd=3
        dup2(fd, 1);   // fd1(stdout)이 foo.txt를 가리킴
                       // 이전 fd1의 refcnt-- (터미널 OFT)
                       // foo.txt OFT의 refcnt++
        close(fd);     // fd3 닫음 (더 이상 불필요)
        execve("ls", args, environ);  // ls 실행 → fd1에 출력 → foo.txt에 쓰임
```

strace로 확인:

```
$ strace -f bash -c "ls > foo.txt" 2>&1 | grep foo.txt
[pid 526] openat(AT_FDCWD, "foo.txt", O_WRONLY|O_CREAT|O_TRUNC) = 3
[pid 526] dup2(3, 1) = 1
[pid 526] close(3) = 0
[pid 526] execve("/usr/bin/ls", ...)
```

복잡한 예제:

```
        // Q: "abcde" 내용의 파일에 대해 출력은?
        int fd1 = open(fname, O_RDONLY);  // fd1→OFT_1 (pos=0)
        int fd2 = open(fname, O_RDONLY);  // fd2→OFT_2 (pos=0) ← 별도 OFT!
        int fd3 = open(fname, O_RDONLY);  // fd3→OFT_3 (pos=0)
        dup2(fd2, fd3);  // fd3→OFT_2 (fd2와 같은 OFT!)
                         // OFT_3는 refcnt=0 → 해제

        read(fd1, &c1, 1);  // OFT_1.pos: 0→1, c1='a'
        read(fd2, &c2, 1);  // OFT_2.pos: 0→1, c2='a'
        read(fd3, &c3, 1);  // OFT_2.pos: 1→2, c3='b' (fd3가 OFT_2 공유!)

        printf("c1=%c, c2=%c, c3=%c\n", c1, c2, c3);
        // 출력: c1=a, c2=a, c3=b
```

Standard I/O vs Unix I/O

Standard I/O (fopen, fprintf 등):

```
FILE* = fd + 내부 버퍼 (8KB)
```

```
버퍼링 효과:
  printf("h"); printf("e"); printf("l"); printf("l"); printf("o");
printf("\n");
  → 실제 write() 시스템 콜은 1번만 호출 (버퍼에 쌓였다가 \n에서 flush)

버퍼 flush 조건:
  1. '\n' 출력 (라인 버퍼링 모드)
  2. fflush(stdout) 명시적 호출
  3. exit() 또는 main return
  4. 버퍼 가득 참

장점: 시스템 콜 횟수 감소 → 성능 향상
단점: 시그널 핸들러 내 사용 불가 (async-signal-unsafe)
     소켓에 사용 불가 (제약 충돌)
```

I/O 함수 선택 가이드:

상황	권장 함수	이유
디스크/터미널 파일	Standard I/O	버퍼링, Short count 자동 처리
시그널 핸들러 내	Unix I/O	async-signal-safe
네트워크 소켓	RIO	Short count 처리 + 소켓 호환
최고 성능 필요	Unix I/O	최소 오버헤드
소켓 + Standard I/O	✗ 금지	스트림/소켓 제약 충돌

Ch14. Debugger & ptrace

GDB는 어떻게 동작하는가?

GDB는 ptrace 시스템 콜을 통해 디버깅 대상 프로세스(tracee)의 메모리와 레지스터를 완전히 제어한다.

ptrace API:

```c
#include <sys/ptrace.h>

long ptrace(enum __ptrace_request request,
            pid_t pid,      // tracee의 PID
            void *addr,     // 메모리 주소
            void *data);    // 데이터
```

ptrace Request 명령 상세

Request	동작	용도
PTRACE_TRACEME	자신을 부모가 trace 하도록 허용	자식 초기화
PTRACE_PEEKTEXT/DATA	Tracee 메모리 word 읽기	코드/데이터 검사
PTRACE_POKETEXT/DATA	Tracee 메모리 word 쓰기	브레이크포인트 삽입
PTRACE_GETREGS	Tracee 레지스터 전체 복사	레지스터 값 읽기
	Tracee 레지스터 전	

PTRACE_SETREGS	체 덮어쓰기	레지스터 조작
PTRACE_CONT	정지된 tracee 재개	계속 실행
PTRACE_SYSCALL	다음 syscall 진입/탈출 시 정지	strace 구현
PTRACE_SINGLESTEP	명령어 1개 실행 후 정지	stepi 구현

GDB 기반 Debugger 전체 동작 흐름

```
GDB (Tracer/부모)                Target (Tracee/자식)
═══════════════════              ═══════════════════

fork()
  └──────────────────────────────→ [자식 프로세스 생성]
                                    PTRACE_TRACEME
                                    → "나를 부모가 trace해도 좋다"
                                    execve("target_program")
                                    → 프로그램 로드
                                    → SIGTRAP 자동 발생! ← execve 후 자
동
                                    → 프로세스 정지

waitpid() ←── SIGTRAP ──────────
→ 첫 SIGTRAP 수신 (execve 완료)

PTRACE_PEEKTEXT(addr)
→ 브레이크포인트 설정할 주소의 원래 바이트 저장

PTRACE_POKETEXT(addr, 0xcc)
→ 해당 주소에 0xcc (int 3) 삽입

PTRACE_CONT ────────────────────→ [실행 재개]
                                   ...
                                   0xcc 명령어 실행
                                   → CPU: SIGTRAP 예외 발생
                                   → 프로세스 정지

waitpid() ←── SIGTRAP ──────────
→ 브레이크포인트 도달 확인

PTRACE_GETREGS
→ 현재 레지스터 값 읽기 (%rip, %rsp, %rax 등)

사용자에게 결과 출력
사용자: "continue" 입력

PTRACE_POKETEXT(addr, orig_byte)
→ 원래 바이트 복원

PTRACE_SETREGS (%rip = addr)
→ PC를 브레이크포인트 주소로 되돌림

PTRACE_SINGLESTEP ──────────────→ [명령어 1개만 실행]
                                   → SIGTRAP 발생

waitpid() ←── SIGTRAP ──────────
```

```
PTRACE_POKETEXT(addr, 0xcc)
→ 브레이크포인트 재삽입

PTRACE_CONT ──────────────────→ [실행 재개]
... 반복 ...
```

Breakpoint 3종류 상세 비교

Software Breakpoint (소프트웨어 브레이크포인트)

```
명령어: int 3 (opcode: 0xcc, 1 byte)
→ CPU가 이 명령어 실행 시 SIGTRAP 예외 발생
→ GDB가 SIGTRAP을 받아 실행 중단

설정 과정:
  1. PTRACE_PEEKTEXT(pid, addr)로 원래 바이트 저장
  2. PTRACE_POKETEXT(pid, addr, 0xcc)로 교체
  3. PTRACE_CONT로 실행 재개

해제 과정 (계속 실행하려면):
  1. PTRACE_POKETEXT(pid, addr, orig_byte)로 복원
  2. PTRACE_SETREGS로 %rip을 addr로 설정 (이미 0xcc 이후를 가리키므로)
  3. PTRACE_SINGLESTEP으로 해당 명령어 실행
  4. PTRACE_POKETEXT(pid, addr, 0xcc)로 재삽입
  5. PTRACE_CONT로 재개

특징:
  + 개수 제한 없음
  + 모든 실행 코드에 설정 가능
  - 코드를 수정해야 함
  - Read-only 코드에는 설정 불가 (권한 필요)
```

Hardware Breakpoint (하드웨어 브레이크포인트)

```
CPU의 디버그 레지스터 활용:
  DR0~DR3: 브레이크포인트 주소 저장 (최대 4개)
  DR7:     각 DR0~DR3의 활성화/조건 제어
           (실행 브레이크 / 읽기 브레이크 / 쓰기 브레이크)

동작:
  CPU가 명령어 실행 전 DR0~DR3와 비교
  일치하면 → Debug Exception → SIGTRAP → GDB 수신

특징:
  + 코드 수정 불필요
  + Read-only 코드에도 설정 가능
  - 최대 4개만 설정 가능 (DR0~DR3)
  - GDB에서 hbreak 명령어로 설정
```

Memory Breakpoint (메모리 브레이크포인트 / Watchpoint)

```
페이지 권한(Permission)을 변경하여 구현:
  해당 페이지를 non-readable/non-executable로 설정
  → 해당 페이지 접근 시 Protection Fault 발생
  → GDB가 Fault를 받아 실행 중단
```

```
용도:
  특정 변수의 값이 변경되는 시점을 찾을 때 (watchpoint)
  메모리 접근 패턴 분석

특징:
  + 코드 수정 불필요
  + 읽기/쓰기 접근 모두 감지 가능
  - 페이지 단위로만 적용 가능 → 같은 페이지의 다른 변수도 걸림
```

strace — System Call 추적기

```
strace: ptrace(PTRACE_SYSCALL)을 이용해 syscall을 추적하는 도구

동작 원리:
  PTRACE_SYSCALL → tracee를 재개하되,
                   다음 syscall 진입 또는 완료 시 자동 정지

strace 내부 루프:
  while (1):
    PTRACE_SYSCALL → 실행 재개
    waitpid() → SIGTRAP 대기

    PTRACE_GETREGS → 레지스터 읽기
    syscall_number = regs.rax
    arg1 = regs.rdi, arg2 = regs.rsi, ...
    "→ syscall_name(arg1, arg2, ...)" 출력

    PTRACE_SYSCALL → 실행 재개 (syscall 실행)
    waitpid() → SIGTRAP 대기 (syscall 완료 후)

    PTRACE_GETREGS → 반환값 읽기 (regs.rax)
    "= return_value" 출력

예시 출력:
$ strace ./hello
execve("./hello", ["hello"], ...) = 0
...
write(1, "hello\n", 6) = 6
...
exit_group(0) = ?
```

GDB 주요 명령어

```
$ gdb ./program              # 디버깅 시작

(gdb) run [args]             # 프로그램 실행
(gdb) break *0x400500        # 주소에 브레이크포인트
(gdb) break main             # 함수명에 브레이크포인트
(gdb) stepi                  # 명령어 1개 실행 (PTRACE_SINGLESTEP)
(gdb) nexti                  # 명령어 1개 실행 (함수 호출은 통과)
(gdb) continue               # 다음 브레이크포인트까지 실행
(PTRACE_CONT)
(gdb) info registers         # 모든 레지스터 값 출력 (PTRACE_GETREGS)
(gdb) set $rax = 42          # 레지스터 값 설정 (PTRACE_SETREGS)
(gdb) x/10x 0x400500         # 메모리 덤프 (PTRACE_PEEKTEXT)
(gdb) set *0x400500 = 0x90   # 메모리 값 설정 (PTRACE_POKETEXT)
(gdb) disassemble main        # 함수 디스어셈블
(gdb) info breakpoints        # 브레이크포인트 목록
```

```
(gdb) delete 1              # 브레이크포인트 삭제
(gdb) watch var             # watchpoint 설정 (Hardware/Memory BP)
```

Ch15. Security Attacks

Cold Boot Attack

배경: 디스크 암호화의 신뢰 모델

```
신뢰 경계 (OS가 신뢰될 때):

CPU Registers  ┐
CPU Cache      ├─ Trusted (암호화 키 저장)
               ┤
DRAM           ├─ ← 문제! 암호화 키가 일시적으로 존재
               ┤
Flash/HDD      ┘  Untrusted (암호화된 데이터)

전원 켜진 상태에서 디스크 암호화를 사용하더라도,
암호화 키는 DRAM에 평문으로 존재해야 한다.
```

공격 원리:

```
DRAM 셀 = 커패시터
  → 전원 공급 시: 주기적 refresh로 데이터 유지
  → 전원 차단 시: 커패시터 방전 → 데이터 소멸

하지만 온도를 낮추면 방전 속도가 극적으로 감소!
  -50°C: 1분 후에도 0.2% 미만만 손실

공격 절차:
  1. 냉각제(액체 질소, 압축 공기)로 DRAM을 빠르게 냉각
  2. 컴퓨터 전원 차단
  3. DIMM 슬롯에서 DRAM 모듈 분리
  4. (냉각 상태 유지하며) 다른 머신에 삽입
  5. 메모리 전체 덤프
  6. 덤프에서 암호화 키 패턴 찾기
  7. 키로 디스크 복호화

실제 공격 성공 사례:
  Windows BitLocker, MacOS FileVault
  Linux dm-crypt, TrueCrypt
```

대응책: TRESOR

```
TRESOR [USENIX Security 2011]:
  Linux 커널 패치
  AES 암호화 키 → CPU 레지스터(DR0~DR3)에만 저장
  DRAM으로 절대 내려가지 않음

원리:
  debug registers는 컨텍스트 스위치 때도 특별 처리
  hibernation 시 레지스터 내용 삭제
  → DRAM에 키가 존재하지 않음 → Cold Boot Attack 불가
```

Side-Channel Attack

정의: 알고리즘 자체의 취약점이 아닌, **구현 방식의 부수 정보**를 이용한 공격 - 타이밍 (Timing) - 전력 소비 (Power consumption) - 전자기 누출 (Electromagnetic leaks) - 음향 (Sound)

Timing Side-Channel Attack

```c
void password_box(char *input) {
    const char *password = "SnuEngEce";
    if (strcmp(password, input) != 0) {
        return ERR;
    }
    return OK;
}
```

공격 원리:

```
strcmp는 첫 번째 불일치 문자에서 즉시 반환:
  "aa..." vs "SnuEngEce" → 첫 글자('a' vs 'S') 불일치 → 즉시 반환
  "Sa..." vs "SnuEngEce" → 첫 글자 일치 → 두 번째 비교
  "Sna.." vs "SnuEngEce" → 두 글자 일치 → 세 번째 비교
  ...

일치하는 앞 글자 수에 비례하여 실행 시간이 증가!

공격:
  time("aa") == time("ba") == ... == time("Ra") < time("Sa")
  → 첫 글자가 'S'일 때 시간이 더 걸림 → 첫 글자 = 'S'
  → 다음: time("Sa") == time("Sb") < time("Sn")
  → 두 번째 글자 = 'n'
  → 반복...

복잡도 비교:
  무작위 추측: 52^9 ≈ 2.3 × 10^15 (사실상 불가능)
  타이밍 공격: 52 × 9 = 468 (매우 쉬움!)
```

대응: Constant-Time 비교 구현

```c
// 항상 같은 시간이 걸리는 비교
int constant_time_compare(const char *a, const char *b, size_t len)
{
    int result = 0;
    for (size_t i = 0; i < len; i++)
        result |= a[i] ^ b[i];  // 모든 바이트 비교, 조기 종료 없음
    return result;
}
```

Cache Side-Channel: Flush+Reload

```
전제: 공격자와 피해자가 공유 메모리를 가짐
      (공유 라이브러리, 파일 mmap 등)

Flush+Reload 3단계:
  1. Flush: clflush로 공유 메모리 캐시 라인 제거
     clflush(&shared_memory[target_addr]);

  2. Wait: 피해자가 공유 메모리에 접근할 때까지 대기
```

```
  3. Reload: 공유 메모리 접근 시간 측정
     t1 = rdtsc();
     temp = shared_memory[target_addr];
     t2 = rdtsc();
     elapsed = t2 - t1;

     elapsed < threshold → Cache Hit → 피해자가 접근함
     elapsed > threshold → Cache Miss → 피해자가 접근 안 함

→ 피해자가 어떤 주소에 접근했는지 추론 가능
→ 주소 = 정보 (암호화 키 비트, 비밀 값 등)
```

Meltdown 공격

배경: Speculative Execution (투기적 실행)

```
현대 CPU는 성능을 위해 명령어를 미리 실행한다:
  분기 결과가 확정되기 전에 한쪽을 미리 실행
  권한 검사가 완료되기 전에 명령어를 미리 실행

미리 실행한 결과가 잘못됐으면 → 되돌림 (revert)
  레지스터: 되돌림
  캐시 상태: !!!되돌리지 않음!!!  ← 이것이 문제
```

공격 코드 (Simplified Meltdown PoC):

```c
int main(void) {
    char user_array[255 * 4096];  // 255 페이지 크기 배열

    // Step 1A: user_array 전체 캐시 비우기
    for (int i = 0; i < 255; i++)
        clflush(&user_array[i * 4096]);

    // Step 1A: 커널 주소 접근 (권한 위반이지만 투기적으로 실행됨)
    char kernel_data = *(char *)0xffffffff81a000e0;
    // → 권한 검사 전에 CPU가 투기적으로 실행
    // → kernel_data에 실제 커널 메모리 값이 들어옴

    // Step 1B: kernel_data를 인덱스로 user_array에 접근
    //          → 해당 캐시 라인이 로드됨 (covert channel)
    maccess(user_array[kernel_data * 4096]);
    // → 예외 처리: kernel_data 레지스터 값은 롤백됨
    //              하지만 캐시 상태는 롤백되지 않음!

    // Step 2: Flush+Reload로 kernel_data 값 복원
    for (int index = 0; index < 255; index++) {
        uint64_t time1 = rdtsc();
        volatile char temp = user_array[index * 4096];
        uint64_t elapsed = rdtsc() - time1;

        if (elapsed < CACHE_HIT_THRESHOLD) {
            // index 번째 캐시 라인이 캐시에 있음
            // → kernel_data == index
            printf("Secret byte: 0x%02x\n", index);
        }
    }
}
```

전체 흐름:

```
1. user_array 캐시 비우기 (clflush)
2. 커널 주소에 투기적 접근
   → CPU: "권한 검사 나중에 할게, 일단 실행"
   → kernel_data = Mem[0xffffffff81a000e0] = 0x42 (예)
3. user_array[0x42 * 4096] 접근
   → 해당 캐시 라인이 로드됨
4. 예외 처리: kernel_data 레지스터 롤백
   → CPU는 권한 위반으로 예외 발생 처리
   → 하지만 캐시에는 user_array[0x42 * 4096]의 흔적이 남아있음!
5. Flush+Reload:
   → user_array[0*4096]: slow (캐시 miss)
   → user_array[1*4096]: slow
   → ...
   → user_array[0x42*4096]: FAST (캐시 hit!) ← kernel_data = 0x42
```

대응책: KPTI (Kernel Page Table Isolation)

```
기존: 유저 모드에서 커널 페이지 테이블이 매핑되어 있음
     (성능을 위해, 하지만 커널 접근은 권한으로 막음)
     → Meltdown: 투기적 실행으로 권한 검사 전에 접근

KPTI 후: 유저 모드에서 커널 페이지 테이블 자체가 없음
         → Meltdown 불가 (페이지 테이블에 없으므로 투기적 실행도 불가)

부작용:
  시스템 콜 시 페이지 테이블 교체 필요 (TLB flush)
  Context Switch 시 페이지 테이블 교체
  → 5~20% 성능 저하
  → 2017년 Linux 커널 메인라인에 병합
```

Rowhammer 공격

DRAM 구조와 취약점:

```
DRAM 물리 구조:
  DIMM → Rank → Bank → Array(Row × Column)

DRAM 셀 = 커패시터
  → 64ms마다 refresh 필요 (방전되므로)
  → 접근 단위: Row (한 Row 전체를 활성화)

문제:
  같은 Row를 반복해서 활성화(hammer)하면
  → 인접한 Row의 커패시터에서 전하 누설 발생
  → 비트 플립 가능!

공격자가 제어하는 메모리     Victim(커널) 메모리
  Row n-1 (aggressor)      Row n (victim)
  Row n+1 (aggressor)

Row n-1과 Row n+1을 반복 접근 → Row n에서 비트 플립 발생
```

두 가지 기술적 도전:

```
도전 1: Cache Bypass (캐시 우회)
  문제: 반복 접근이 캐시에 캐시되면 DRAM에 도달하지 않음
       → Row activation 발생 안 함 → 비트 플립 없음

  해결: clflush 명령어로 캐시 강제 비우기
  while (1):
```

```
    access(row_A)        # DRAM Row activation
    access(row_B)        # DRAM Row activation
    clflush(row_A)       # 캐시에서 제거 (다음 접근 시 DRAM 직접 접근)
    clflush(row_B)


도전 2: Address Information (인접 Row 주소 파악)
  문제: 가상 주소로는 어느 물리 Row가 인접한지 알 수 없음

  해결: VA 하위 12bit = PA 하위 12bit
  (4KB 페이지 내 오프셋은 VA=PA)
  → 대량의 메모리를 할당하고 주소 패턴을 분석하여
    동일한 Bank의 인접 Row를 역공학으로 파악
```

두 가지 실제 공격:

공격 1: NaCl Sandbox Escape

```
NaCl (Native Client): Chrome의 샌드박스
  = 안전한 x86 부분 집합만 실행 (Software Fault Isolation)
  = 검증기(validator)가 코드를 사전 검사
  = 간접 점프는 32-byte 정렬된 주소만 허용
  = 단, clflush는 허용됨 (sandbox 내에서 실행 가능)

공격 아이디어:
  NaCl 검증을 통과한 코드를 Rowhammer로 비트 플립
  → 검증된 안전한 코드가 위험한 코드로 변환
  → 점프 정렬 제약 우회 가능
  → Sandbox 탈출

공격 절차:
  1. 취약한 명령어 시퀀스를 NEXE에 대량 복사 (Spray)
  2. clflush + 반복 접근 (Rowhammer)
  3. 비트 플립 발생 확인 (코드 읽기 가능)
  4. 원하는 비트 플립 없으면 → 2로 돌아감
  5. 성공 시 → Sandbox 탈출!
```

공격 2: Linux 커널 권한 상승

```
목표: PTE(Page Table Entry)의 비트를 플립
      → 비특권 프로세스가 커널 메모리에 접근 가능

PTE 구조 (64-bit):
  비트 63:    XD (No-Execute)
  비트 51-12: PPN (Physical Page Number) ← 20bits = 31% 확률
  ...
  비트 1:     RW (Read/Write) ← 1bit = 2% 확률 (1/64)
  비트 0:     P (Present)

RW bit 플립: 읽기 전용 → 읽기/쓰기 권한 획득
PPN bit 플립: 다른 물리 페이지에 매핑 → 커널 메모리 접근 가능

공격 절차:
  1. 대량의 메모리 할당 (2GB 등)
  2. Rowhammer: 반복 접근하며 비트 플립 발생 위치 탐색
  3. 해당 메모리를 커널에 반환 (free)
  4. 커널이 반환된 메모리를 PTE로 사용하도록 유도
     (대량의 주소 공간 매핑 → 커널이 PTE 배열 확장)
  5. Rowhammer: 비트 플립 발생!
     → PTE.RW = 1 → 쓰기 권한 획득
     → PTE.PPN 변조 → 커널 물리 페이지 직접 접근
```

6. 커널 메모리 읽기/쓰기 → 권한 상승 완료

핵심 통계:
PTE 64 bits 중:
RW bit: 1bit → 플립 시 2% 확률로 쓰기 권한 획득
PPN: 20bits → 플립 시 31% 확률로 다른 물리 페이지 접근

공격 종합 비교

공격	공격 대상	핵심 원리	대응책
Cold Boot	DRAM 암호화 키	냉각으로 데이터 소멸 지연	TRESOR
Timing	비밀 정보	실행 시간 차이 측정	Constant-time 구현
Cache SCing	접근 패턴	Flush+Reload 시간 측정	캐시 파티셔
Meltdown	커널 메모리	Speculative exec + 캐시 (5~20% 저하)	KPTI
Rowhammer	DRAM 비트	인접 Row 반복 → 비트 플립	ECC DRAM, TRR

공통점:
소프트웨어 버그가 없어도 공격 가능 (하드웨어/구현 결함 이용)
기존 보안 경계(주소 공간 격리, 권한 검사)를 우회